Smart Parking, Traffic Enforcement & Citizen Reporting System

Research blueprint: references, similar systems, CV pipeline, stack, demo and build plan

11 October 2026

Part 0. Executive blueprint
- The one-line pitch
- Five things that matter most
- Recommended stack (all free)
- Indian-conditions warnings
- Demo flow (6 minutes, detailed script in Part 3, section 5)
- Build order
- Judge Q&A ammunition

Part 1. Computer vision: models, datasets, repos, pipeline
- 1. Detection models
- 2. Tracking for dwell time
- 3. Datasets
- 4. Repos and papers
- 5. Demo footage
- 6. ANPR (future eChallan link) and privacy
- 7. Practical pitfalls and fixes
- 8. Recommended pipeline
- Gaps / unverified

Part 2. Similar systems, official references, judge Q&A
- Project: Smart Parking, Traffic Enforcement & Citizen Reporting System (India)
- 0. TL;DR (what matters for the pitch)
- 1. Official Indian references
- 2. Existing Indian systems: what they do and their gaps
- 3. Global systems
- 4. Similar hackathon projects
- 5. Research papers & datasets
- 6. Gaps / differentiators this project can claim
- 7. Likely judge questions, with suggested answers
- 8. Corrections to make in the one-pager
- 9. Not covered or unverified

Part 3. Stack, data model, algorithms, APIs, demo, build plan
- 0. TL;DR (the decisions)
- 1. Architecture & stack
- 2. Data model (PostgreSQL + PostGIS)
- 3. Algorithms
- 4. APIs
- 5. Demo script (6 minutes) + pitch structure
- 6. Build plan (team of 3–4, 24–48 h)
- 7. Open-source starters, libraries & references
- 8. Gaps & caveats (be honest in Q&A)

Part 0. Executive blueprint

The one-line pitch

Turn the CCTV traffic police already own into a live parking map for citizens and a single enforcement console for police: zone occupancy, one-click open/close, grace-period illegal-parking flags that an officer verifies, and citizen complaints merged into the same incident.

Five things that matter most

1. **It's legal and it's precedented.** CMVR Rule 167A (Aug 2021, under MV Act s.136A) allows challans from electronic-enforcement footage for parking at an unauthorised location (s.122), with plate photo, date/time/place, section and a 65B certificate. Towing (s.127) needs an officer, so your officer-verification step is legally required, not decoration. Dubai's Parkin uses the same "AI flags, inspector reviews, grace period" design.
2. **eChallan has no public API.** Access goes through NIC's NAPIX gateway with MoRTH approval and a CERT-In audit. Build a "challan-ready draft" in eChallan's field format and say real integration goes through state police credentials.
3. **Fix two names in the one-pager.** Delhi's Traffic Sentinel is now **Traffic Prahari** (1 Sep 2024). Bengaluru's **Public Eye** was folded into **BTP ASTraM** (Mar 2025). TMICC guidance was issued under MoUD (MoHUA's old name); pair it with IMAF 2.0.
4. **Your differentiator.** No Indian system combines CCTV availability map + police open/close controls + grace-period flags with human review + merged citizen reports. Indian smart-parking projects (Chennai GCC, Mumbai BMC, NDMC) failed on operations and hardware contracts; you need no new hardware.
5. **Closest rival:** SIH 2023 winner "iPark" (NMIMS Indore, MoHUA problem SIH1515) used CCTV for availability, reservations and pricing. You go further with enforcement, police controls and complaint merging.

Recommended stack (all free)

- **CV worker:** Python, Ultralytics **YOLO26n** on CPU (export to OpenVINO/ONNX), ByteTrack (`model.track(persist=True)`), Roboflow `supervision` PolygonZone, 1–2 fps sampling, custom dwell/grace-period state machine (sketch in Part 1).
- **Fake CCTV:** MediaMTX + `ffmpeg -re -stream_loop -1 -i clip.mp4 -c copy -f rtsp rtsp://localhost:8554/cam1`.
- **Backend:** FastAPI + native WebSockets. **DB:** PostgreSQL + PostGIS (Docker locally, Neon free in cloud; avoid Render free Postgres, it expires after 30 days).
- **Frontend:** React + Vite + TS + Tailwind + shadcn; Leaflet + OSM, leaflet.heat, h3-js; `vite-plugin-pwa` for the citizen reporting app; Google Maps deep link for navigation (`travelmode=two-wheeler`).
- **Demo:** run everything locally via `docker compose`, phones on the laptop hotspot; cloud copy as backup.

Indian-conditions warnings

- COCO models have **no auto-rickshaw class**. Fine-tune on DriveIndia (67k images, YOLO format) or IDD-Detection, plus a few hundred frames from your own pole-height angle.
- Rows of two-wheelers: estimate occupancy by zone coverage, not individual counts. Count per zone, not per marked slot.
- Plates are usually unreadable on pole cameras; pitch ANPR (fast-alpr) as a future module.
- Privacy: DPDP Act s.17(1)(c) largely exempts police offence detection, but s.8(5) security safeguards apply. Blur faces/plates in public views, store counts not raw video.

Demo flow (6 minutes, detailed script in Part 3, section 5)

Problem slide → CCTV window counting “Beach Road 16/20” → citizen phone map (amber, “4 left”, “Usually full 6–9 PM”, Navigate) → police dashboard alert at 17/20 → officer closes street for festival, phone turns grey instantly → one car stays, dwell timer turns red after 25 s demo grace → second phone files “double parking” report, merged into same case, priority jumps → officer verifies, “Send to eChallan (draft)” → hotspots + forecast → impact/cost slide. Every risky step has a “Simulate vehicle” fallback through the same code path, plus a recorded backup video.

Build order

Hour 0–2 freeze the API contract → CV worker on sample video + seeded zones → backend status/alerts/WS → citizen map → police dashboard → complaint merge + priority → hotspots/forecast → rehearse. Hard gate: end-to-end working locally by hour 24.

Judge Q&A ammunition

Kerala’s AI cameras logged 20.4 lakh detections in month 1 but issued about 1.04 lakh challans because verification was the bottleneck (your priority queue + merging addresses this). Israel’s citizen Road Guards programme rejected 80%+ of reports (your CCTV corroboration addresses this). Full list of 13 likely questions with answers in Part 2, section 7.

Part 1: Computer vision research. **Part 2:** Similar systems, official references, judge Q&A. **Part 3:** Stack, schema, algorithms, APIs, demo script, build plan.

Links marked [E] were seen in search excerpts only; open them before quoting a number on a slide.

Part 1. Computer vision: models, datasets, repos, pipeline

Researched 11 Oct 2026. Every URL below was opened (fetched) or returned in a search result during this session. Pages marked "(search result)" exist but blocked direct fetch (Roboflow, Pexels search pages and similar return 403 to scripts), so their details come from search snippets. GitHub star counts were read from the live repo pages on 11 Oct 2026.

Short version: Use **Ultralytics YOLO26n**, exported to **OpenVINO** (or ONNX), with Ultralytics' built-in tracker (`model.track(persist=True)`, ByteTrack). Put **supervision** **`PolygonZone`** on top to count spaces in each zone, and add your own small dwell-time state machine for no-parking zones, which keeps a vehicle's timer running through short occlusions. Sample at **1–2 fps**. Fine-tune on an Indian dataset (IDD-Detection or DriveIndia) if you want an `autorickshaw` class, because COCO doesn't have one. The violation logic below has been tested on synthetic detections.

---

1. Detection models

Ultralytics YOLO (current state, Oct 2026)

- **YOLO26** is Ultralytics' newest release (Jan 2026; YOLO27 is announced as "coming soon"). It has an NMS-free end-to-end head (`nms=False`) and no DFL. The paper reports **up to 43% faster CPU ONNX inference for YOLO26n than YOLO11n**. https://docs.ultralytics.com/models/yolo26/
- COCO detection, 640 px, CPU ONNX latency (Ultralytics' tables):

Model	mAP50-95	CPU ONNX ms	Params
YOLO26n	40.9	**38.9**	2.4 M
YOLO26s	48.6	87.2	9.5 M
YOLO26m	53.1	220.0	20.4 M
YOLO11n	39.5	56.1	2.6 M
YOLO11s	47.0	90.0	9.4 M

Sources: https://docs.ultralytics.com/models/yolo26/ , https://docs.ultralytics.com/models/yolo11/ . YOLOv8 still works with the same API, but there's no reason to start a new project on it. - **What this means for a laptop:** YOLO26n uses about 40 ms/frame on CPU, roughly 25 fps. At 1–2 fps sampling, one laptop can run several camera feeds. Use YOLO26s if small two-wheelers get missed. - **License:** Ultralytics models and code are **AGPL-3.0**, with an Enterprise licence also offered. That's fine for an open hackathon repo. A closed commercial product would need the Enterprise licence. https://docs.ultralytics.com/models/yolo26/

Built-in "Solutions" that match this project

- **`solutions.ParkingPtsSelection()`** opens a Tkinter GUI: you click polygons on a frame and it saves `bounding_boxes.json`. **`solutions.ParkingManagement(model="yolo26n.pt", json_file="bounding_boxes.json")`** then marks each slot occupied or free and shows lot occupancy. It accepts tracker args (`tracker`, `conf`, `classes`, `device`, `imgsz`). On **Fedora**, install Tk first: `sudo dnf install python3-tkinter`. https://docs.ultralytics.com/guides/parking-management/
    - It works per slot, so it fits marked bays. For "Beach Road: 20 spaces, 16 occupied", where bays aren't marked, a single zone polygon with a capacity number is simpler (see the pipeline below).

- **`solutions.RegionCounter(region={name: points,...})`** gives live occupancy per named polygon. It counts objects whose **box centre** lies inside, resets the count every frame, and returns `region_counts`. https://docs.ultralytics.com/guides/region-counting/
- **`ObjectCounter`** counts objects crossing a line or region (in/out totals), which is useful for counting entries and exits. **TrackZone** tracks objects inside a single zone. **Object Blurring** is also built in, for privacy. https://docs.ultralytics.com/guides/region-counting/ (cross-links), https://docs.ultralytics.com/guides/trackzone/ , https://docs.ultralytics.com/guides/object-blurring/

Faster CPU inference: OpenVINO / ONNX

- Export with `model.export(format="openvino")` (add `quantize=8` for INT8 with calibration data) and load it with `YOLO("yolo26n_openvino_model/")`. Ultralytics says this gives **up to 3× CPU speedup** on Intel and can also use Intel iGPU/NPU (`device="intel:gpu"`). https://docs.ultralytics.com/integrations/openvino/
  - Benchmarks on that page: on Core Ultra 7 155H, YOLO26n PyTorch-CPU runs at 33.2 ms, OpenVINO FP32 at 9.1 ms and INT8 at 5.8 ms, with mAP 0.4765 → 0.4573 for INT8. The page doesn't say clearly whether each OpenVINO row ran on CPU, iGPU or NPU, so treat these as best-case figures.
- On an AMD laptop, use ONNX (`format="onnx"`) with onnxruntime. OpenVINO is tuned for Intel.

Other models

- **RT-DETR** (in Ultralytics as `RTDETR("rtdetr-l.pt")`): 53.0 AP and 114 FPS on a **T4 GPU**. Ultralytics notes it "excels on accelerated backends like CUDA with TensorRT". It's heavy on CPU, so skip it without a GPU. https://docs.ultralytics.com/models/rtdetr/
- **RF-DETR** (Roboflow, 9.8k★, https://github.com/roboflow/rf-detr) comes in nano/small sizes. The supervision time-in-zone example ships an `rfdetr_file_example.py` with `--device cpu`. It's a good alternative if you want Apache-style licensing instead of AGPL. (README: https://github.com/roboflow/supervision/tree/develop/examples/time_in_zone)
- Other small or edge options listed by Ultralytics: YOLOE (open-vocabulary, so you can prompt "auto rickshaw" as text), plus NCNN/MNN/LiteRT exports for a Raspberry Pi or phone. https://docs.ultralytics.com/models/yolo26/ , https://docs.ultralytics.com/integrations/openvino/ (nav)

---

2. Tracking for dwell time

- **Ultralytics built-in trackers** (`model.track(frame, persist=True, tracker=...)`): BoT-SORT, ByteTrack, OC-SORT, Deep OC-SORT, FastTracker, and **TrackTrack (now the default)**. https://docs.ultralytics.com/modes/track/
  - Ultralytics' own guidance: **ByteTrack** is the "fastest, simplest baseline" for **static cameras**, which is what CCTV is. BoT-SORT adds camera-motion compensation and optional ReID (set `gmc_method: none` for a static camera). FastTracker handles frequent partial overlap.
  - The key setting is **`track_buffer`**, the number of frames a lost track stays alive. At 1–2 fps it means seconds of occlusion it can survive. Raise it, or handle gaps in your own logic (as the sketch below does).
  - Use `persist=True` only on consecutive frames from **one** stream. Keep one model and tracker instance per camera.
- Original trackers: ByteTrack (https://github.com/FoundationVision/ByteTrack, 6.7k★, ECCV 2022), BoT-SORT (https://github.com/NirAharon/BoT-SORT, 1.6k★), OC-SORT (https://github.com/noahcao/OC_SORT, 1.2k★). BoxMOT (https://github.com/mikel-brostrom/boxmot, 8.3k★) collects plug-in trackers. Roboflow `trackers` (https://github.com/roboflow/trackers, 3.9k★; docs https://trackers.roboflow.com) is what the newest supervision example imports (`from trackers import ByteTrackTracker`).

- **supervision (Roboflow, 51k★, MIT)**, https://github.com/roboflow/supervision
  - `sv.PolygonZone(polygon, triggering_anchors=(sv.Position.BOTTOM_CENTER,))` → `zone.trigger(detections)` returns a boolean per detection and sets `zone.current_count`. The default anchor is **BOTTOM_CENTER**, which suits vehicles because it's roughly where the tyres touch the road. Note from the docs: it is "anchor-based, not a true geometric box/polygon intersection". https://supervision.roboflow.com/latest/detection/tools/polygon_zone/
  - **Time-in-zone example** (`examples/time_in_zone`): YOLO + ByteTrack + PolygonZone + `FPSBasedTimer` / `ClockBasedTimer`, a `draw_zones.py` polygon tool that saves JSON, a `stream_from_file.py` script that fakes an RTSP camera with Docker, and a traffic example using classes `[2,5,6,7]`. https://github.com/roboflow/supervision/tree/develop/examples/time_in_zone
  - **Caveat:** their timer (https://raw.githubusercontent.com/roboflow/supervision/develop/examples/time_in_zone/utils/timers.py) only maps tracker_id to first frame. It never resets when a car moves and has no tolerance for lost tracks. **If the tracker issues a new ID after an occlusion, the dwell clock restarts.** That's why you need your own grace-period logic.
- Installed in this sandbox: `supervision==0.30.9` still exposes `sv.ByteTrack` (checked 11 Oct 2026).

Grace-period logic (what the sketch implements)

1. A vehicle's **bottom-centre** enters a no-parking polygon → start `first_seen`.
2. If the anchor moves more than `move_tol_px` (e.g. 25 px) from where it entered, it's moving or manoeuvring → restart the timer. This separates "stopped" from "slow traffic".
3. If it isn't seen for up to `lost_tolerance_s` (e.g. 15 s, covering a bus passing in front), keep the timer. Only drop it after that.
4. When `now - first_seen ≥ grace_s` (e.g. 120 s), raise **one** alert with status `PENDING_OFFICER_REVIEW`. Attach a snapshot or crop; an officer confirms. Never auto-challan.
5. Optional: before alerting, re-identify after ID switches by IoU against the last box of recently lost tracks in the same zone. This catches the "new ID, same parked car" case.
6. Optional: show three stages, Monitoring → Warning → Violation. One existing repo does Warning at 60 s and Violation at 300 s (Dard1ka/Ai-Ilegal-Parking, below).

---

3. Datasets

Parking-occupancy datasets

Dataset	What	Link
PKLot (de Almeida et al., ESWA 2015)	12,417 images at 1280×720 from 2 lots and 3 views (UFPR04, UFPR05, PUCPR) in sunny, cloudy and rainy weather; 695,899 labelled space crops (occupied/empty); CC BY 4.0	https://web.inf.ufpr.br/vri/databases/parking-lot-database/ ; paper PDF https://www.inf.ufpr.br/lesoliveira/download/ESWA2015.pdf
PKLot on Roboflow	12,416 images converted to boxes around the rotated slot rectangles; COCO/YOLO export	https://public.roboflow.com/object-detection/pklot (search result) ; HF copy https://huggingface.co/datasets/Voxel51/PKLot (search result)
CNRPark+EXT (Amato et al., ESWA 2017)	~150,000 labelled patches, 164 spaces, 9 cameras, Nov 2015–Feb 2016, with weather changes, shadows and occlusion by trees and lampposts	http://cnrpark.it/ ; downloads and trained mAlexNet weights at https://github.com/fabiocarrara/deep-parking (108★, release "archive")
ACPDS (Marek 2021)	293 GoPro images from about 10 m height; each lot appears in only one split, so it tests generalisation; baseline 98% on **unseen lots**; MIT	https://github.com/martin-marek/parking-space-occupancy (51★) ; https://arxiv.org/abs/2107.12207

These are mostly images of car-only Western or Brazilian lots shot from high up. They're useful for a slot-classifier fallback, but they don't contain Indian street scenes.

Indian vehicle datasets (autos and two-wheelers)

Dataset	What	Link
IDD (India Driving Dataset), IIIT-H + Intel	Segmentation set: 10,004 images, 34 classes, Hyderabad and Bangalore, mostly 1080p, front dashcam. **IDD-Detection** has about 46.6k images with `autorickshaw`, `motorcycle`, `rider`, `vehicle fallback` classes, and even `license plate`	https://idd.insaan.iiit.ac.in/ ; details https://idd.insaan.iiit.ac.in/dataset/details ; dataset list https://insaan.iiit.ac.in/datasets/ (search result)
DriveIndia (Kumar et al., 2025)	66,986 images in **YOLO format**, 24 classes including Auto-rickshaw, Motorcycle, Pushcart, Tractor, Police vehicle; fog, rain and night; 3,400+ km. Their YOLO baseline scores Auto-rickshaw at mAP50 0.94	https://arxiv.org/abs/2507.19912
IRUVD	Indian Road User Vehicle Dataset: pedestrian plus 13 vehicle types, urban and rural (2★)	https://github.com/IRUVD/IRUVD
ThirdEye Labs Indian road dataset	Annotated Indian road footage; 12 classes including `autorickshaw`, `animal`, `vehicle fallback`; labels made with YOLO11m fine-tuned on IDD + ByteTrack	https://huggingface.co/datasets/thirdeyelabs/indian-road-dataset
Roboflow Universe	Many small community sets, e.g. "Indian Vehicle Dataset" (classes car, truck, bus, bike, ambulance, auto-rikshaw, police vehicle; CC BY 4.0), "Vehicle Detection Indian v1" (~46k images, includes Rickshaw), plus `class:autorickshaw` search	https://universe.roboflow.com/hf-riaag/indian-vehicle-dataset , https://universe.roboflow.com/indianvehicledatasetv1/vehicle-detection-indian-v1 , https://universe.roboflow.com/search?q=class%3Aautorickshaw (all search results)

Why Indian traffic is hard for detection

- **COCO has no autorickshaw class.** A COCO model labels autos as car or motorcycle, or misses them. The ThirdEye dataset card says the same about BDD100K: those models "cannot emit" autorickshaw at all. https://huggingface.co/datasets/thirdeyelabs/indian-road-dataset
- Two-wheelers are **small, packed in dense rows and overlapping**. That breaks per-vehicle counts, so in two-wheeler parking measure **area occupied** rather than counting units, or use `imgsz=960/1280`.
- Rare classes score far lower: in DriveIndia's baseline, Pushcart is 0.39 and Police vehicle 0.51 mAP50, against about 0.94 for Car and Auto. https://arxiv.org/abs/2507.19912
- Almost all Indian datasets are **dashcam viewpoints**. CCTV looks down from a pole, so fine-tune on a few hundred frames from your own camera angle.

Recommendation: for the hackathon, start with COCO YOLO26n, classes `[1,2,3,5,7]` (bicycle, car, motorcycle, bus, truck); it will get autos partly right as car or motorcycle. If time allows, fine-tune YOLO26n for about 30–50 epochs on a Colab/Kaggle GPU using DriveIndia or IDD-Detection plus 200–300 of your own CCTV frames, with classes {car, motorcycle, autorickshaw, bus, truck, bicycle}.

---

4. Repos and papers

Parking occupancy repos

Repo	★	What
https://github.com/ultralytics/ultralytics	62.4k	Includes `ParkingManagement` and `RegionCounter`, the most useful starting point
https://github.com/roboflow/supervision	51.2k	`PolygonZone` plus the time-in-zone example
https://github.com/olgarose/ParkingLot	224	Classic OpenCV (no DL) per-slot occupancy from a static camera
https://github.com/visualbuffer/parkingslot	115	Finds slots automatically with Mask R-CNN/YOLO, then classifies occupancy
https://github.com/fabiocarrara/deep-parking	108	mAlexNet / CNRPark code and weights
https://github.com/martin-marek/parking-space-occupancy	51	ACPDS baseline
https://github.com/computervisioneng/parking-space-counter	40	Tutorial-level slot counter
https://github.com/danielbob32/ParkingSpace	31	YOLOv11 + RTSP, handles areas **without marked bays** (probability map), which suits Indian streets
https://github.com/thebkht/smart-parking-system	6	Edge ACPDS classifier (YOLOv8n-cls, 97.7% on unseen lots) that sends JSON instead of video

Illegal / no-parking violation repos (all small; use for ideas, not as code to depend on)

Repo	★	What
https://github.com/prakharninja0927/illegal-Parking-Detection	12	YOLOv4 + designated-zone check
https://github.com/Kevinjoythomas/Illegal-Parking-Detection	1	YOLO + DeepSORT, threshold time in a restricted zone, saves a JSON log and JPEG evidence
https://github.com/Dard1ka/Ai-Ilegal-Parking	0	YOLOv8 + **BoT-SORT**, ID re-linking across occlusions (up to 80 s), multi-zone ROI dashboard, **3-stage alert Monitoring→Warning(60 s)→Violation(300 s)**, extra model to spot warning triangles (breakdown exception)
https://github.com/MThabsheer7/traffic-violation-system	0	**YOLO26n INT8 via OpenVINO** on the edge, illegal-parking dwell + wrong-way, FastAPI + SQLite + React, yellow at >33% of dwell threshold and red at violation
https://github.com/mamoonzaheer/smart-parking-violation-detection (search result)	0	YOLOv8 vehicle + plate, EasyOCR, SORT, FastAPI, SQLite
https://github.com/algyonfaras/parking-violation-detection-yolov8 (search result)	0	Thesis: ROI + stopping duration, Flask dashboard, CSV export

Repo	★	What
https://github.com/Starcoder10/Illegal-Parking-detection (search result)	0	YOLOv8 + MOG2 background subtraction + centroid stability check before the timer

Key papers

- **Amato et al., “Deep learning for decentralized parking lot occupancy detection”**, Expert Systems with Applications 72:327–334 (2017). Introduces mAlexNet and CNRPark-EXT; occupancy of 50 spaces in about 15 s on a Raspberry Pi. https://www.sciencedirect.com/science/article/pii/S095741741630598X ; preprint https://openportal.isti.cnr.it/data/2017/366883/2017_366883.preprint.pdf
- **de Almeida et al., “PKLot – A robust dataset for parking lot classification”**, ESWA 42(11) (2015). https://www.inf.ufpr.br/lesoliveira/download/ESWA2015.pdf
- **Marek, “Image-Based Parking Space Occupancy Classification: Dataset and Baseline”** (2021, ACPDS). https://arxiv.org/abs/2107.12207
- **Xie et al., “Real-Time Illegal Parking Detection System Based on Deep Learning”** (2017). SSD detection, then tracking and motion analysis in a **manually marked ROI**, with a timer started once the vehicle is stationary; reports 99% accuracy at 25 FPS. This is the closest academic match to this project. https://arxiv.org/abs/1710.02546
- **“Parking Time Violation Tracking Using YOLOv8 and Tracking Algorithms”**, Sensors 23(13):5843 (2023). YOLOv8 + DeepSORT/OC-SORT + time boundaries on CCTV, reporting MOTA per scene. https://www.mdpi.com/1424-8220/23/13/5843 (search result; MDPI blocks scripts)
- Classical baselines on the UK **i-LIDS “Parked Vehicle”** set: GMM + Kalman with a 60 s rule (AICCSA 2017) https://www.computer.org/csdl/proceedings-article/aiccsa/2017/3581a840/12OmNz2TCDM ; DSP system (AVSS 2007) https://www.computer.org/csdl/proceedings-article/avss/2007/04425320/12OmNqGA54i (search results)
- **NVIDIA AI City Challenge, traffic anomaly track** (2018–2021): Iowa DOT highway video at 800×410, about 15 min per clip; anomalies are **crashes and stalled vehicles**. The 2021 Track 4 data is the only set still listed as downloadable. One 2021 method treats a vehicle stopped longer than a signal cycle as an anomaly, the same dwell idea used here. https://www.aicitychallenge.org/2021-challenge-tracks ; https://www.aicitychallenge.org/2019-data-sets ; Box-level tube tracking paper https://openaccess.thecvf.com/content/CVPR2021W/AICity/papers/Wu_Box-Level_Tube_Tracking_and_Refinement_for_Vehicles_Anomaly_Detection_CVPRW_2021_paper.pdf . The 2026 Track 3 is now VLM-style reasoning: https://huggingface.co/datasets/nvidia/PhysicalAI-Traffic-Anomaly-Reasoning

---

5. Demo footage

- **Ultralytics sample assets** (direct links, HTTP 200 checked):
  - https://github.com/ultralytics/assets/releases/download/v0.0.0/solution_ci_parking_demo.mp4 , the parking video Ultralytics’ CI uses for `ParkingManagement`
  - https://github.com/ultralytics/assets/releases/download/v0.0.0/solution_ci_parking_areas.json , its matching slot polygons
  - https://github.com/ultralytics/assets/releases/download/v0.0.0/parking-management-compressed.mp4
  - Licence: the ultralytics/assets repo is AGPL-3.0.
- **Pexels**: the free licence allows use and modification with no attribution required. You may not sell unaltered copies, show identifiable people in a bad light, or imply endorsement. https://www.pexels.com/license/ . Search “parking lot cctv” (about 50k videos, mostly aerial or drone, with a few fixed-camera clips such as “night view of busy parking lot outside retail store” and “a footage of a covered parking lot”): https://www.pexels.com/search/videos/parking%20lot%20cctv/ . Also try “india traffic”, “indian street”, “auto rickshaw”.

- **Pixabay**: free, no attribution, modification allowed. Prohibited: selling unmodified ("standalone") copies, misleading use, and immoral or illegal use of recognisable people. https://pixabay.com/service/license-summary/
- **YouTube Indian street and CCTV videos:** the Terms of Service forbid downloading "except as expressly authorized by the Service" and forbid collecting information that could identify a person, such as faces. https://www.youtube.com/static?template=terms . Supervision's example ships a YouTube download script, but downloading still breaks the ToS. For a public demo, prefer Pexels/Pixabay or **your own footage** (a phone on a tripod overlooking a road near campus, like a pole camera), and blur plates and faces.
- **Best demo:** record 10–15 min yourself on a static tripod from a balcony or footbridge over a street with a "No Parking" stretch. Stage one vehicle parking past the grace period. This gives an Indian context (autos, two-wheelers) and avoids licensing problems.
- Live-feed mock: supervision's `stream_from_file.py` loops a file as `rtsp://localhost:8554/live0.stream` (needs Docker/Podman on Fedora). https://github.com/roboflow/supervision/tree/develop/examples/time_in_zone

---

6. ANPR (future eChallan link) and privacy

Open-source ANPR

- **fast-alpr** (https://github.com/ankandrew/fast-alpr, 829★, MIT): YOLOv9-t plate detector + `fast-plate-ocr`, ONNX Runtime, `pip install fast-alpr[onnx]` (CPU) or `[onnx-openvino]`. The OCR is pluggable. **The default "global" OCR is not tuned for Indian plates**, so check it against two-line and painted plates.
- **fast-plate-ocr** (https://github.com/ankandrew/fast-plate-ocr, 773★), which you can fine-tune on Indian plates.
- **PaddleOCR** (https://github.com/PaddlePaddle/PaddleOCR, 90.9k★) and **EasyOCR** (https://github.com/JaidedAI/EasyOCR, 30.1k★) are general OCR fallbacks. **Awiros/anpr-ocr** is a 37M-parameter PP-OCRv5 model fine-tuned on 558,767 Indian plate samples covering single- and **dual-row** plates; check its licence before use. https://huggingface.co/Awiros/anpr-ocr
- OpenALPR (https://github.com/openalpr/openalpr, 11.5k★) has been largely inactive since around 2024 (last push Jan 2024), so it isn't recommended.
- Indian plate data: "Indian Licence Plate Dataset in the wild" (16,192 images, 21,683 plates; the authors note Indian non-standard formats break OCR) https://arxiv.org/abs/2111.06054 , repo https://github.com/sanchit2843/Indian_LPR (52★; **dataset not yet public due to legal review**). DataCluster Labs set of about 6,000 images from 20+ states: https://github.com/datacluster-labs/Indian-Licence-Plate-Image-Dataset . A Kaggle set of 1.5k images (search result): https://www.kaggle.com/datasets/saisirishan/indian-vehicle-dataset
- **Practical limit:** plates on a pole CCTV are often too small to read. ANPR usually needs a dedicated ANPR camera or a crop from a high-resolution frame. Pitch it as a **future module**.

Legal hook for enforcement (strong pitch point)

- **CMVR Rule 167A** (G.S.R. 575(E), 11 Aug 2021), "Electronic Monitoring and Enforcement of Road Safety", lists CCTV and ANPR as "electronic enforcement devices". **Footage with an electronic stamp of location, date and time can be used to issue a challan.** A device must carry a state-signed approval certificate renewed every year. The e-challan must include photographic evidence showing the offence and the plate, plus date, time and place. https://www.pib.gov.in/PressReleasePage.aspx?PRID=1747285 ; rule text https://indiacode.ecourtsindia.com/rules/17th-gsr-575-e-11-august-2021-67003722/ (search result)
- MoRTH issued an **SOP on 28 Oct 2025** for implementing Rules 167/167A (site selection, devices, calibration, standard e-challan process), and a CMVR amendment on 20 Jan 2026 gave it legal

backing. Lok Sabha answer: https://sansad.in/getFile/loksabhaquestions/annex/187/AU4543_ulL9Jn.pdf?source=pqals (search result)
- Design implication: store a timestamped evidence frame, camera ID and location with every alert, and keep the **officer verification step**. That matches how Rule 167A expects evidence to look.

DPDP Act 2023 / DPDP Rules 2025

- The DPDP Rules were notified on 14 Nov 2025. Most compliance rules (3, 5–16, 22, 23) take effect **18 months later, around May 2027**. https://www.pib.gov.in/PressReleasePage.aspx?PRID=2190655 ; phase-in summary https://www.pgak.co.in/insights/is-ai-cctv-legal-in-india-dpdp-act (search result)
- **Section 17(1)(c)** exempts processing for preventing, detecting, investigating or prosecuting offences from most duties, **but not from s.8(5) reasonable security safeguards**. https://www.ahuva.in/resources/dpdp-act-cctv-access-control-data/
- Rule 8(3) requires keeping processing logs for at least one year, unless another law says otherwise. https://cadp.in/resources/official-texts/dpdp-rules-2025/ (search result)
- What to build in for privacy:
  - Analyse only vehicles: no face recognition, and blur faces with Ultralytics Object Blurring.
  - Store **counts and alerts, not raw video**.
  - Keep evidence crops only for flagged events and delete them after the officer decides.
  - Use role-based access and audit logs.
  - Show plate text only to officers, masked in the public app.
  - For **citizen reports**, collect only what's needed (photo, GPS, time) and blur bystanders' faces and plates in public views.

---

7. Practical pitfalls and fixes

Pitfall	Fix
Occlusion (buses or trucks blocking parked cars; dense rows of two-wheelers)	Use the bottom-centre anchor; set a lost-track tolerance (10–20 s) before resetting the dwell; raise the tracker's `track_buffer`; smooth occupancy with a 5–10 sample median or majority vote (danielbob32 and other repos do this). For two-wheeler zones, estimate occupancy by **area** (union of boxes ÷ zone area) instead of counting.
Perspective (far vehicles are tiny; a box can spill into the next zone)	Draw zones on the road surface and test the **bottom-centre** point, not the box centre. Use a larger `imgsz` (960–1280) or SAHI-style slicing (`sv.InferenceSlicer`) for far zones. For real-world distances, use a homography to a top-down view (`cv2.getPerspectiveTransform`).
Camera angle	Pole or overhead views work best. Low angles cause heavy overlap (the parking-occupancy literature reports the same for wide-angle and near-ground views: https://arxiv.org/pdf/2306.04288).
Night, rain, glare, IR mode	Indian CCTV switches to greyscale IR at night. Fine-tune on night frames (DriveIndia has low-light images), lower `conf` to around 0.2 at night, try CLAHE preprocessing, and expect more misses. Headlight glare causes false positives, so require N consecutive detections.
Polygon setup per camera	One JSON per camera ID: `{zone_id, type: parking/no_parking, capacity, grace_s, polygon}`. Draw it with `ParkingPtsSelection()` or supervision's `draw_zones.py`. Redraw when a camera is bumped (detect drift by comparing against a saved reference frame with ORB features).
Frame rate	Parking changes slowly: **1 fps is enough for occupancy, 2 fps is safer for tracking**, since IoU trackers need

Pitfall	Fix
	overlap between frames. At 0.5 fps or below, IDs swap often, so use your own IoU-to-last-box rematch. Use `sv.get_video_frames_generator(stride=...)`, or for RTSP read every frame but run the model on every Nth (`cap.grab()` to skip).
"Stopped" vs "parked" (traffic jam, red signal, auto picking up passengers)	Require the vehicle to be stationary (anchor drift below a pixel threshold) **and** still for the grace period; skip zones on active lanes during signal red phases; officer review is the final filter.
Class confusion (auto read as car or motorcycle)	Use class-agnostic "vehicle" for occupancy; fine-tune for the auto class if fees differ.
CPU budget	YOLO26n + OpenVINO/ONNX, `classes=` filter, 640 px, 1–2 fps → about 10 cameras per laptop is plausible (estimate, not measured).

8. Recommended pipeline

```
RTSP / video file ──► frame sampler (1–2 fps) ──► YOLO26n (OpenVINO/ONNX, classes=vehicles)
                      └─► tracker (Ultralytics ByteTrack, persist=True, one per camera)
                          ├─► Parking zones: sv.PolygonZone.trigger → occupied/capacity → median
smoothing
                          │        └─► publish {zone, occupied, free, ts}  (FastAPI/WebSocket → map/
app)
                          └─► No-parking zones: dwell state machine (grace, move tolerance, lost
tolerance)
                                   └─► alert {cam, zone, track, dwell, evidence.jpg, ts} → officer
review queue
                                            └─► (future) ANPR crop → fast-alpr → eChallan draft
```

Libraries: `ultralytics` (YOLO26n + trackers + export), `supervision` (PolygonZone, frame generator, annotators), `opencv-python`, `openvino` or `onnxruntime`, and FastAPI + SQLite/Postgres for the backend. On Fedora: `sudo dnf install python3-tkinter` for the polygon GUI, then `pip install ultralytics supervision "ultralytics[export-openvino]"`.

Minimal code sketch (tested logic)

Full file: `/workspace/projects/54f01675-75ef-404e-bfd4-383eb438499f/work/parking_cv_sketch.py`. I tested the `NoParkingZone` dwell logic and the `occupancy` function on synthetic detections with supervision 0.30.9. A parked car was flagged at exactly the 10 s grace period, a passing car was not flagged, and a 3 s occlusion gap did not reset the timer. The YOLO/`model.track` part was **not** run here because no weights or video were available in the sandbox.

```python
import json, time
from dataclasses import dataclass, field
import numpy as np, supervision as sv

VEHICLE_CLASSES = [1, 2, 3, 5, 7]  # COCO bicycle, car, motorcycle, bus, truck (no autorickshaw in
        COCO)

@dataclass
class Dwell:
    first_seen: float; last_seen: float; anchor: np.ndarray; flagged: bool = False

@dataclass
class NoParkingZone:
    name: str
    zone: sv.PolygonZone
```

```
    grace_s: float = 120.0          # grace period before "possible violation"
    lost_tolerance_s: float = 15.0  # survive occlusion / brief track loss
    move_tol_px: float = 25.0       # moved more than this => not parked, restart
    tracks: dict = field(default_factory=dict)

    def update(self, dets: sv.Detections, t: float) -> list[dict]:
        alerts = []
        inside = dets[self.zone.trigger(dets)]
        anchors = inside.get_anchors_coordinates(sv.Position.BOTTOM_CENTER)
        for tid, xy, cls in zip(inside.tracker_id, anchors, inside.class_id):
            d = self.tracks.get(int(tid))
            if d is None or np.linalg.norm(xy - d.anchor) > self.move_tol_px:
                self.tracks[int(tid)] = Dwell(t, t, xy); continue
            d.last_seen = t
            if not d.flagged and t - d.first_seen >= self.grace_s:
                d.flagged = True
                alerts.append({"zone": self.name, "track_id": int(tid), "class_id": int(cls),
                               "dwell_s": round(t - d.first_seen, 1), "status": "PENDING_OFFICER_REVI
        EW"})
        self.tracks = {k: v for k, v in self.tracks.items() if t - v.last_seen <= self.lost_tolerance_
        s}
        return alerts

def occupancy(zone, dets, capacity):
    n = int(zone.trigger(dets).sum())
    return {"occupied": min(n, capacity), "capacity": capacity, "free": max(capacity - n, 0)}

def main(video, zones_path, model_path="yolo26n.pt", sample_fps=2.0):
    from ultralytics import YOLO
    cfg = json.load(open(zones_path))
    parking = {k: (sv.PolygonZone(np.array(v["polygon"])), v["capacity"]) for k, v in cfg["parking"].
        items()}
    nopark = [NoParkingZone(k, sv.PolygonZone(np.array(v["polygon"])), v.get("grace_s", 120))
              for k, v in cfg["no_parking"].items()]
    model = YOLO(model_path)                        # or "yolo26n_openvino_model/"
    info = sv.VideoInfo.from_video_path(video)
    stride = max(int(round(info.fps / sample_fps)), 1)
    for i, frame in enumerate(sv.get_video_frames_generator(video, stride=stride)):
        t = i * stride / info.fps                   # use time.time() for live RTSP
        r = model.track(frame, persist=True, classes=VEHICLE_CLASSES, conf=0.3,
                        tracker="bytetrack.yaml", verbose=False)[0]
        dets = sv.Detections.from_ultralytics(r)
        if dets.tracker_id is None: continue
        snapshot = {n: occupancy(z, dets, c) for n, (z, c) in parking.items()}   # e.g. Beach Road
        16/20
        for npz in nopark:
            for a in npz.update(dets, t):
                print("ALERT", a)                   # save crop + push to officer queue
```

`zones.json` example:

```
{"parking":    {"Beach Road P1": {"capacity": 20, "polygon": [[100,400],[700,400],[700,600],
        [100,600]]}},
 "no_parking": {"Beach Road NP1": {"grace_s": 120, "polygon": [[720,380],[1000,380],[1000,620],[720,62
        0]]}}}
```

Build order for the hackathon

1. Get the sketch running on `solution_ci_parking_demo.mp4`, then on your own phone footage.
2. Export to OpenVINO or ONNX, and measure fps on your laptop: `model.benchmark()` or simply time the loop.
3. Add a dashboard: FastAPI + WebSocket, a map of zones with free/occupied counts, and an officer queue with evidence crops and Accept/Reject buttons.

4. Optional extras: fine-tune for autos, blur plates and faces, add a citizen-report upload, and stub ANPR with fast-alpr.

Gaps / unverified

- I couldn’t fetch Roboflow Universe, Pexels search pages for India, or the MDPI page directly (403 to scripts); their details come from search snippets.
- The CPU fps on your laptop is not measured. The Ultralytics figures come from their hardware: Xeon for the ONNX table, Core Ultra for OpenVINO.
- I didn’t verify whether IDD-Detection requires a login to download (the IDD site has a login and download page).
- Exact licences of the Roboflow community datasets vary by dataset; check each one before use.

Part 2. Similar systems, official references, judge Q&A

Project: Smart Parking, Traffic Enforcement & Citizen Reporting System (India)

Compiled 2026-10-11. Every URL below was returned by a search or opened during this research.
Source marker: `[F]` = page opened and read in full; `[E]` = seen only as a search-result excerpt (verify before quoting on a slide).

---

0. TL;DR (what matters for the pitch)

1. **The law already allows camera-based parking challans.** CMVR Rule 167A (Aug 2021, under MV Act s.136A) lists "stopping or parking vehicle at an unauthorised location (section 122)" as an offence that can be challaned from electronic-enforcement footage, **provided** the challan carries photo evidence with plate, date/time/place, the section violated, and a Sec 65B electronic-evidence certificate, and the device has a yearly state approval certificate. This is your strongest legal answer.
2. **eChallan already has an ITMS integration path and a citizen-reporting module ("Citizen Sentinel").** Your "future link to eChallan" is realistic: NIC's eChallan advertises "integration with ITMS devices", an "ITMS Challan Web Application", "Traffic Sentinel Mobile App" and "Citizen Sentinel". Data access is through NIC's **NAPIX API gateway**, approved by MoRTH, CERT-In audit required, PII unmasked only for police.
3. **Two names in your one-pager may be out of date:** Delhi's "Traffic Sentinel" was **relaunched as "Traffic Prahari" on 1 Sep 2024**; Bengaluru's **Public Eye was phased out in Mar 2025** and folded into **BTP ASTraM**. Use the current names.
4. **Indian smart-parking projects keep failing on operations, not tech:** Chennai's GCC Smart Parking (2019–2025) collapsed with contractor turnover; Mumbai BMC's tender got no bidders across 5 extensions; NDMC re-tendered in 2025; Pune's mechanised Vertipark is being demolished. **Re-using existing police CCTV (no new hardware, no per-bay sensors, no fee-collection contract) is a genuine differentiator.**
5. **Nobody in India combines all four:** (a) public availability map from existing CCTV, (b) police-controlled open/close/temporary restrictions, (c) grace-period flagging with officer verification, (d) citizen complaints with duplicate merging + hotspots. Each exists in isolation.
6. **Dubai's Parkin uses the same "AI flags → human inspector reviews → grace period" design** (May 2026). Cite it to show your human-in-the-loop workflow is international best practice.

---

1. Official Indian references

1.1 MoHUA / Smart Cities: TMICC & ICCC guidance

Document	What it says that's useful	Source
TMICC guidance document (Traffic Management & Information Control Centre), prepared under MoUD (now MoHUA) GEF-Sustainable Urban Transport Project with World Bank/GEF	Reference guide "to assist various government organisations and public authorities in India embarking on the process of establishing TMICCs… planning, designing, procuring, establishing,	https://smartnet.niua.org/content/ddcc994d-1091-48d7-b614-a7684cc6499f `[E]` (page fetch failed)

Document	What it says that's useful	Source
	managing and monitoring". **This is the doc your one-pager cites.** Note: published by MoUD, the ministry's name before it became MoHUA in 2017.	
ICCC Maturity Assessment Framework (IMAF) toolkit, 2018/2020	Defines city mobility services as Transit, Traffic Management and **"City Parking Solutions: management of public parking spaces in terms of usage as well as revenue collection."** Lists "Smart Parking Management System" as a use case.	https://smartnet.niua.org/sites/default/files/resources/iccc_maturity_assessment_framework_toolkit_vf211218.pdf [E]; https://smartnet.niua.org/sites/default/files/draft_part_a-_imaf_2020.pdf [E]
IMAF 2.0	10 pillars (infrastructure, use cases, cybersecurity, data lifecycle, governance, business plan, inter-departmental coordination, stakeholder engagement, innovation). Use-case list includes **"Traffic enforcement using Intelligent Traffic Management System"**.	https://iccc.niua.org/assets/frontend/flip/pdf/IMAF2.0.pdf [E]
PIB/MoHUA: ICCCs in all 100 smart cities	30 of 100 ICCCs work on traffic (ATCS, ITMS, RLVD, ANPR).	https://pib.gov.in/PressReleaseIframePage.aspx?PRID=1907135 [E]
NIUA ICCC use case: Bhopal smart parking → ICCC	ICCC receives real-time parking feeds from a Smart Parking Command Centre, maps them on GIS, feeds analytics and can push commands/alerts back. **Good architecture precedent**: your system should plug into the ICCC the same way.	https://iccc.niua.org/iccc/sector/use-case/815e6212def15fe76ed27cec7a393d59 [E]
Smart Cities Mission page (MoHUA)	All 100 smart cities have operational ICCCs used for traffic regulation and crowd management at high-footfall events.	https://mohua.gov.in/offerings/schemes-and-services/details/smart-cities-czM5cTMtQWa [E]
Parliament answer (Dec 2025)	MoHUA has advised cities on **financial and functional sustainability** of ICCCs. Useful framing: your system adds a use case to an asset cities are already told to sweat.	https://sansad.in/getFile/annex/269/AU861_38NDk7.pdf?source=pqars [E]
IRC draft Smart Parking Management guideline (IRC ITS G-7 committee)	A draft national guideline for smart parking deployment exists.	https://cdnc.heyzine.com/files/uploaded/d1be790bc56dac7df400626fba47754324da5433-1.pdf [E]
IRC SP:12-2015 Guidelines for Parking Facilities in Urban Areas	Recommends parking-availability info via mobile apps and centralized systems, plus surveillance cameras.	https://law.resource.org/pub/in/bis/irc/irc.gov.in.sp.012.2015.pdf [E]

1.2 Parking policy

- **National Urban Transport Policy (NUTP) 2006**: parking fees should reflect the land value occupied; electronic metering is encouraged; legislation should "prevent the use of the right of way on road systems for parking purposes." https://mohua.gov.in/upload/uploadfiles/files/TransportPolicy.pdf [E]
- **NUTP 2014 revision**: says cities should move "from the conventional approach to using parking as a demand management tool." https://www.changing-transport.org/wp-content/uploads/E_K_NUMP_India_2014_EN.pdf [E]
- **National TOD Policy 2017**: no on-street parking within 100 m of transit; heavy penalty for unregulated parking; time-of-day pricing. https://www.mohua.gov.in/static/uploads/2025/10/ba0cf0a6a17909cf36099ae402f2ab63.pdf [E]
- **Parking Area Management Plan (PAMP) toolkit**, from GIZ and MoHUA's SMART-SUT programme, for Bhubaneswar. https://crdf.org.in/project/parking-area-management-plan-pamp-for-bhubaneswar-toolkit [E]
- **Gap I could not close:** I found **no single "National Urban Parking Policy" document from MoHUA**. National parking guidance sits in NUTP 2006/2014, the TOD policy and the IRC guidelines. Parking policy itself is a city/state subject (Delhi, Rajkot and Bhubaneswar each have their own). On the one-pager, say "NUTP and MoHUA parking guidance", not "National Parking Policy".

1.3 MoRTH / NIC eChallan & Parivahan

- **What eChallan is:** An Android app for enforcement officers plus a web portal, integrated with **Vahan** (vehicle registration) and **Sarathi** (driving licences). https://parivahan.gov.in/sites/default/files/FAQDOCS/Echallan/eChallan-brochure-ver-1.0.pdf [E]; user manual https://parivahan.gov.in/sites/default/files/FAQDOCS/Echallan/echallan-user-manual.pdf [E]
- **NIC eChallan brochure** [F]: "integration with ITMS devices and also with virtual court". Components: eChallan Mobile App, eChallan Web App, **ITMS Challan Web Application**, Violation on Camera App, **Traffic Sentinel Mobile App**, Tow Away Mobile App, **Citizen Sentinel**. Features include "Integrated with ITMS/Smart City Project", geo-tagging, photo/video evidence and back-end integration with Vahan/Sarathi. The ITMS flow includes an NIC-hosted "ITMS challan / notice server" that validates traffic rules and ANPR-converted data. https://cdnbbsr.s3waas.gov.in/s3dcf6070a4ab7f3afbfd2809173e0824b/uploads/2025/09/20250901904555723.pdf
- **How ITMS → eChallan works in practice (Gujarat, NIC Informatics Apr 2025):** Cameras capture the violation, registration details are fetched from VAHAN and licence details from SARATHI, and the eChallan is sent to the mobile number linked to the vehicle. Unpaid challans go to Virtual Traffic Court after 90 days. https://informatics.nic.in/files/websites/april-2025/echallan.php [E]
- **Public API? Not open.** The **MoRTH Policy for Data Sharing from the National Transport Repository** [F] says:
    - API-based sharing is the preferred mode, "published via the NIC API Exchange Gateway (NAPIX)", secured with secret keys, IP whitelisting (max 4 IPs) and yearly renewal.
    - Requests go to Director (MVL), MoRTH, signed by an officer of at least ADGP rank (enforcement agencies) or Administrative Secretary rank (states). The applicant needs a CERT-In empanelled security audit, logs kept for 1 year, and data stored in India.
    - **PII is masked in API responses except for police/law enforcement.**
    - The policy explicitly covers "third-party for e-Challan implementations": states running their own eChallan versions can onboard third parties for select Vahan/Sarathi/eChallan parameters.
    - Citizens get only masked lookups (3/day).
    - **Implication:** a hackathon prototype should **mock** the eChallan hand-off (export a challan-ready evidence packet in eChallan's field format) and say production integration goes through the state traffic police's NAPIX credentials.
    - https://parivahan.gov.in/sites/default/files/policy/data-sharing-policy.pdf

- **eChallan data fields** (from that policy's Annexure VIII, useful for designing your evidence packet): challan number, date-time, location, **lat-long**, police station/circle, officer, offences list, amount, status, vehicle reg no and class, owner/driver, DL number, court details.
- **NextGen eChallan portal** (MH, JK and MZ already migrated): https://echallan.parivahan.nic.in/ [E]; main portal https://echallan.parivahan.gov.in/ [E]

1.4 Motor Vehicles Act: illegal parking provisions

Provision	Content	Source
s.122 Leaving vehicle in dangerous position	Vehicle must not be left at rest on a public place so as to "cause or likely to cause danger, obstruction or undue inconvenience".	https://indiankanoon.org/doc/497562/ [E]
s.127 Removal of abandoned/ unattended vehicles [F]	(1) Vehicle abandoned or unattended **10+ hours**, "or is parked in a place where parking is legally prohibited": towing or wheel-clamping may be authorised by **a police officer in uniform having jurisdiction**. (3) Owner pays towing costs plus any other penalty. **This is why your design needs officer verification before tow or clamp.**	https://indiacode.ecourtsindia.com/mv-act/section/127/
s.177 General penalty	Raised by the 2019 amendment to up to ₹500 for a first offence and ₹1,500 for subsequent ones.	https://narnaul.haryanapolice.gov.in/writereaddata/Document/Motor_vehicel_section_&_fine.pdf [E]
s.122/127 r/w 177	Typical charge for improper/ obstructive parking. Jharkhand lists ₹100/₹300 (pre-2019 schedule); **Hyderabad lists ₹1,000 for illegal parking at No-Parking boards (MV Act 122, 177)**. State schedules differ.	https://echallan.jhpolice.gov.in/document/offences_penalties_ENG.pdf [E]; https://echallan.tspolice.gov.in/htp/mvact.html [E]
s.201 Obstruction to free flow	Disabled vehicle obstructing traffic: up to ₹50/hour plus towing charges.	https://vidhijudicial.com/sec-177-to-210-(chapter-xiii-offences,-penalties-and-procedure)-the-motor-vehicles-act,-1988.html [E]
s.136A + CMVR Rule 167A (G.S.R. 575(E), 11 Aug 2021) [F]	Electronic enforcement devices (CCTV, ANPR, dashcams, body cams) may be used. Footage with an electronic location/date/time stamp can be used to challan **"stopping or parking vehicle at an unauthorised location (section 122)"**. An auto-generated e-challan must carry (i) clear photo of offence and plate, (ii) device measurement, (iii) date/time/place, (iv) the provision violated, (v) a **Sec 65B Indian Evidence Act certificate**. Devices go at critical junctions in 1M+ cities and the 132 listed cities.	PIB: https://www.pib.gov.in/PressReleasePage.aspx?PRID=1747285
Rule 167A(1) and (9)	Device needs an approval certificate from a state authority, **renewed yearly**. **Notice must be sent within 15 days**. The record must be stored until the challan is disposed of (or until appeals end).	https://indiacode.ecourtsindia.com/rules/17th-gsr-575-e-11-august-2021-67003722/ [E]
Rule 167(5)		

Provision	Content	Source
	Challans must be disposed of within 90 days.	https://sansad.in/getFile/loksabhaquestions/annex/186/AU1928_q14MUv.pdf?source=pqals [E]
2026 draft/amendment (G.S.R. 48(E), 20 Jan 2026)	Challan by electronic means within **3 days**; offline capture allowed with later upload. Verify before quoting.	https://mvd.kerala.gov.in/sites/default/files/Downloads/G.S.R.48%28E%29%20dt%2020-01-2026.pdf [E]
Citizen reports	Delhi's app states that date, time, place, registration number and violation type are "mandatory to issue a notice u/s 133 M.V. Act".	https://play.google.com/store/apps/details?hl=en_IN&id=com.dtp.trafficsentinel [E]

Note (not verified in this research, from background knowledge): the Indian Evidence Act 1872 has been replaced by the **Bharatiya Sakshya Adhiniyam 2023**, whose Section 63 corresponds to the old Section 65B. Mention this if a judge is legally minded.

1.5 DPDP Act 2023: privacy relevance

- **s.17(1)(c)** [F]: Chapter II (except **s.8(1) and 8(5)**), Chapter III and s.16 "shall not apply where… personal data is processed in the interest of prevention, detection, investigation or prosecution of any offence or contravention of any law". Police enforcement processing is therefore largely exempt, **but the duty to protect data with reasonable security safeguards (s.8(5)) still applies**. https://indiacode.ecourtsindia.com/digital-personal-data-protection-act-2023/section/17/
- **s.17(4)**: for processing by the State, the erasure obligations s.8(7) and s.12(3) don't apply.
- **s.17(2)(b)**: research and statistical processing is exempt if no decision is taken about an individual. This covers aggregated hotspot and forecast analytics.
- **DPDP Rules 2025 notified 14 Nov 2025**, making the Act fully operational. https://pib.gov.in/PressReleasePage.aspx?PRID=2190014 [E]
- **Where DPDP does bite on this project:**
    - 1. **Citizen complainants' data** (phone, GPS, photos) is ordinary personal data and needs notice and purpose limitation.
    - 1. **Bystanders' faces and plates on the public map** must not be shown. Publish only occupancy counts.
    - 1. MoRTH's data-sharing policy makes any NTR data recipient a Data Fiduciary, with liability under Chapter VIII.
- International analogy: the UK ICO treats a number plate as personal data when processed to identify a person, e.g. to serve a parking fine. https://ico.org.uk/… (ANPR guidance) [E]

---

2. Existing Indian systems: what they do and their gaps

System	What it does	Gap vs this project	Source
BTP ASTraM (Bengaluru; engine since Jan 2024, citizen app since Jan 2025)	5 modules [F]: congestion monitoring and next-day prediction (alerts every 15 min), incident-reporting BOT and map API, ambulance tracking (ePath), event calendar with traffic simulation, dashboard/analytics/workflow. App: live traffic, violation and accident reporting, challan payment, SOS.	**No parking-availability layer and no camera-based parking detection.** Citizen reports are not merged or clustered. Event simulation exists but parking can't be opened or closed per event.	https://btp.karnataka.gov.in/209/astram/en [F]; https://www.thehindu.com/news/cities/bangalore/btp-astram-app-a-one-stop-platform-for-all-commuter-needs-launched/article69155838.ece [E]; https://www.thehindu.com/news/cities/bangalore/college-students-to-help-monitor-traffic-with-new-police-marshals-feature-on-astram-app/article70201941.ece [E]

System	What it does	Gap vs this project	Source
	Oct 2025: “Police Marshals” feature for college students.		
Public Eye (BTP + iChangeMyCity, 2016–2025)	Photo/video reports in 13 categories incl. No Parking, Footpath Parking, Wrong Parking. BTP said action within 48 h.	**Phased out 27 Mar 2025**, replaced by ASTraM. No duplicate merging.	https://www.ichangemycity.com/publiceye-faq [E]; https://www.thehindu.com/news/cities/bangalore/publiceye-app-of-bengaluru-traffic-police-phased-out/article69382959.ece [E]
Delhi Traffic Sentinel → Traffic Prahari (2015, relaunched 1 Sep 2024)	Citizen photo/video reports with GPS, date and time auto-captured. Points per violation (e.g. wrong-side 5, triple riding 2). Monthly cash rewards of ₹50k/25k/15k/10k to the top 4. 3.72 lakh reports by 2018. 10,114 complaints in the first ~2 months after relaunch.	Gamified incentives invite duplicates and spam. App Store reviews complain of a “black hole” with no feedback. **No dedup, no hotspot map for citizens, no parking availability.**	https://staging.parivahan.gov.in/delhitrafficpolice/en/traffic-prahari [F]; https://www.hindustantimes.com/cities/delhi-news/delhi-traffic-police-to-relaunch-upgraded-traffic-sentinel-app-on-september-1-101724953103933.html [E]; https://delhipolice.gov.in/doc/Best_practices-English.pdf [E]; https://apps.apple.com/in/app/traffic-prahari/id1071515704 [E]
mParivahan “Citizen Sentinel” (NIC; Odisha, Chhattisgarh, Kerala from Oct 2024)	Citizen uploads violation photo/video (≤8 MB). MVD verifies and an eChallan goes to the owner. Launch demo: a car in a no-parking zone was challaned under “obstructive parking”.	**Shows the citizen → verify → eChallan pipeline already exists nationally.** Your contribution is the layer before it: dedup, clustering, prioritisation and CCTV corroboration.	https://www.thehindu.com/news/national/kerala/kerala-launches-citizen-sentinel-app-to-keep-tabs-on-traffic-rule-violations/article68768294.ece [E]; https://timesofindia.indiatimes.com/city/kochi/citizens-now-empowered-to-report-traffic-rule-violations-with-new-mparivahan-feature/articleshow/114355658.cms [E]
Kerala AI cameras (Safe Kerala)	726 AI cameras. First month: 20.4 lakh detections, of which 7.42 lakh were verified, 1.78 lakh uploaded to the transport system, 1.29 lakh approved and 1.04 lakh challans posted. A district monitoring committee was set up “to ensure innocent individuals do not pay fines”.	**Shows the verification bottleneck:** only ~5% of detections became challans in month 1. Covers helmet, seatbelt and phone use, **not parking**.	https://www.indiatoday.in/india/story/ai-camera-captures-over-20-lakh-traffic-violations-in-a-month-in-kerala-2402373-2023-07-05 [F]; cost/revenue (₹9.40 cr spent, ₹59.46 cr fined): https://www.thehindu.com/news/national/kerala/over-59-crore-charged-as-fine-by-ai-camera-driven-enforcement-across-kerala/article68277409.ece [E]
Kerala Police Pol-App / Traffic Offence Reporting / MVD “Third Eye” WhatsApp	Pol-App (TCS, 27 services) includes offence reporting with photos. A separate Traffic Offence Reporting app exists. MVD has run a WhatsApp number since 2014.	Fragmented channels, no geo-dedup.	https://keralapolice.gov.in/page/mobile-apps [E]; https://apps.apple.com/us/app/traffic-offence-reporting/id1439762285 [E]; https://www.newindianexpress.com/states/kerala/2014/Nov/04/now-whatsapp-your-grievance-to-mvd-678518.html [E]
Hyderabad / Telangana	32 AI cameras since Apr 2026: 6.34 lakh challans from 3 Jubilee Hills cameras in ~4 months, mostly helmet, wrong-side and phone use. Mobile “robo	Parking enforcement is still manual or mobile-cam based. Availability is limited to pilot lots.	https://hyderabadmail.com/jubilee-hills-ai-cameras-6-34-lakh-traffic-challans/ [E]; https://propakistani.pk/2026/07/28/hyderabad-traffic-police-introduce-robo-cams-for-e-challans/ [E] (secondary source; prefer an Indian outlet); https://timesofindia.indiatimes.com/city/

System	What it does	Gap vs this project	Source
	cams" for wrong parking (Jul 2026). "ROPE" drives against obstructive parking. IIIT-H INAI's **VIOLA** puts dashcams on police/tow vehicles and feeds evidence into eChallan. GHMC pilot: smart parking at 14 Khairatabad locations with sensors and AI cameras.		hyderabad/will-use-ai-to-streamline-traffic-says-hyderabad-police-commissioner-vc-sajjanar/articleshow/128760930.cms [E]; https://www.newindianexpress.com/cities/hyderabad/2026/Oct/01/out-of-box-hyderabad-how-inai-is-using-ai-to-catch-traffic-violations-2 [E]; https://www.thehindu.com/news/cities/Hyderabad/smart-parking-facilities-to-come-up-at-14-locations-in-hyderabad/article70639849.ece [E]
Chennai: GCC Smart Parking (2019–2025)	App with slot sensors and cameras, booking, ANPR e-receipts; ~9,200 functional slots in 2021. In 2019 GCC chose cameras over per-slot sensors ("one camera can capture ten slots"). **Failed**: Toorq contract ended 2024; TEXCO terminated 20 Jul 2025 over an overcharging scandal; reverted to manual POS. New PPP tender planned for 20 locations by end-2026. GCC's internal "GCC Parking" app is monitored at the ICCC.	**Lesson: tech died because of the fee-collection contract model.** Cameras-over-sensors was already the city's preference.	https://1001smartcities.org/projects/chennai-smart-parking/ [F]; https://www.thehindu.com/news/cities/chennai/smart-parking-system-a-solution/article28119959.ece [E]; https://timesofindia.indiatimes.com/city/chennai/chennai-corporation-to-roll-out-smart-parking-in-20-areas/articleshow/128865306.cms [E]; https://www.newindianexpress.com/states/tamil-nadu/2025/Mar/20/track-occupancy-details-of-parking-slots-through-app-soon [E]
NDMC Smart Parking (New Delhi)	87 lots (5,392 4W + 2,237 2W spots). Sensor-based app with real-time availability, entry/exit and **"parking violations"**, flexible tariffs for "special events". 2023: 97 lots, 8,044 vehicles, revenue up from ₹20 lakh to ₹90 lakh. Booking in the NDMC-311 app. Re-tendered 2025 as an Intelligent Integrated Parking System for 152–155 lots over 10 years.	Off-street lots only, per-slot sensors, operator-dependent. Not linked to police enforcement on streets.	https://smartcity.ndmc.gov.in/content/projects/project-details/smart-parking [F]; https://government.economictimes.indiatimes.com/news/governance/ndmcs-command-and-control-centre-is-third-eye-for-coordinating-g-20-summit/103304300 [E]; https://timesofindia.indiatimes.com/city/delhi/ndmc-renews-bid-for-smart-parking-management-across-155-facilities/articleshow/121523588.cms [E]
Mumbai BMC	Smart-parking app tender: no bids after 5 extensions (2024–26). Re-scoped as one contractor for hardware and software (₹3 cr dashboard). Mumbai Traffic Police app supports citizen	Procurement failure. Proposed per-bay sensors make it unviable.	https://www.hindustantimes.com/cities/mumbai-news/bmc-redraws-smart-parking-plan-after-failed-tender-bids-101776713231562.html [E]; https://play.google.com/store/apps/details?id=com.sparken.mum.policealert&hl=en [E]

System	What it does	Gap vs this project	Source
	reports, but reviews call the reporting UX poor.		
Pune / Pimpri-Chinchwad	Pune's 20-storey Vertipark (2007) defunct since ~2012 and slated for demolition (Jul 2026). PCMC smart parking reported as never properly started.	Shows the hardware-heavy failure mode.	https://indianexpress.com/article/cities/pune/pune-pmc-civic-body-mechanised-parking-facility-dismantle-jm-road-10802958/ [E]; https://en.mahaenews.com/city-news/pimpri-chinchwads-smart-city-slow-unplanned-chaotic-administration/ [E]
Indore Smart City	Planned sensors and cameras for parking availability with VMS and app; 7,200 ECS of smart-parking features in the ABD area; MLCP smart parking at 3 locations completed.	Lot-level, not street enforcement.	https://www.smartcityindore.org/pan-city-initiative [E]; https://www.smartcityindore.org/completed-project [E]
Puducherry Smart City (relevant locally)	ICCC (₹63 cr work order 2022). RailTel's ₹170.11 cr ICCC contract (Jan 2023) **includes on-street parking management and ITMS**. Traffic police installing 20 smart signals with ANPR and RLVD cameras (Aug 2024). MLCP tenders at Old Jail / Old Port.	**Puducherry is a natural pilot city:** the ANPR/CCTV and ICCC are coming, and parking is still manual.	https://pondicherrysmartcity.in/view-project.php?id=MTA5 [E]; https://government.economictimes.indiatimes.com/news/governance/integrated-command-control-center-for-puducherry-smart-city-to-be-built-by-railtel/96986507 [E]; https://www.thehindu.com/news/cities/puducherry/traffic-police-embark-on-plan-to-undertake-itms-in-puducherry/article68501711.ece [E]
Get My Parking (GMP)	Bengaluru SaaS. Retrofits LPR and gates; white-label municipal app; pilots in Jaipur, Vijayawada and Kharghar; Hyderabad Metro (24 stations); claims data on 9,000 lots in 50 cities. Says it can give enforcement officials real-time parking and violation data.	Lot/gate-centric and commercial. No police restriction controls, no citizen reporting.	https://www.getmyparking.com/municipality-sector.php [E]; https://www.parking.net/parking-news/get-my-parking/solutions-powering-the-smart-parking-revolution-in-india [E]
Park+	RFID/ANPR access for societies, malls and corporate parks (40+ cities, 10k+ societies). FASTag parking at MahaKumbh 2025.	Private premises only; no on-street or police role.	https://www.linkedin.com/posts/parkplus_kanpur-activity-7444629513143070720-xGJE [E]; https://www.motownindia.com/InNews/In%20the%20News/1919/Park-launches-MahaKumbhs-first-FASTag-enabled-Parking-Management-System [E]
MyParkings	Not researched. No source found in this run.	n/a	n/a

3. Global systems

System	Key facts	Takeaway	Source
SFpark (San Francisco, 2011–13)	In-ground sensors plus demand-responsive pricing (60–80% occupancy target, rates adjusted ~every 8 weeks). Time at target occupancy +31% (control +6%). Time "too full" −16% in pilot areas vs +51% in control. Search time −43%, VMT −30%, **citations −23%**. Academic estimate: cruising cut ~50% (Millard-Ball et al.). Chatman & Manville dissent on availability gains. **The sensors were costly and short-lived**, so SFMTA built a sensor-independent model from meter payments.	Occupancy data plus pricing works, but per-bay sensors don't last. Supports your "use existing cameras" approach.	https://www.sfmta.com/sites/default/files/reports-and-documents/2018/08/sfpark_pilot_overview.pdf [E]; https://www.sfmta.com/blog/san-francisco-adopts-demand-responsive-pricing-program-make-parking-easier [E]; https://www.sciencedirect.com/science/article/abs/pii/S0965856414000470 [E]; https://www.itskrs.its.dot.gov/2016-b01101 [E]
Cleverciti (Munich)	Overhead optical sensors on lampposts, **up to 100 spaces per sensor**, 3-second updates. Edge processing: "the only data that leaves a sensor are the GPS coordinates", so no video leaves the device (privacy). Also flags vehicles in prohibited	Closest commercial analogue to your camera approach. Copy its privacy-by-edge design.	https://www.cleverciti.com/en/solutions [E]; https://www.cleverciti.com/en/resources/blog/how-does-smart-parking-work [E]

System	Key facts	Takeaway	Source
	zones such as hydrants.		
Hikvision / Bosch	Hikvision sells parking-space-detection cameras (fixed-bay and free-area modes) and on-street ANPR poles. Bosch made in-ground sensors; Cocoparks reports that Bosch and Nedap had abandoned ground sensors by 2024.	The industry is moving from ground sensors to cameras.	https://www.hikvision.com/content/dam/hikvision/eu/support/brochures/vertical-solution-brochure/2022-Parking-Solution-Brochure_EU_HR.pdf [E]; https://cocoparks.io/en-us/guides/complete-guide-to-parking-sensors-for-smart-parking-and-curb-management/ [E]
Parkopedia	70–90M spaces in ~90 countries. Predicts availability with models even where there are no sensors. Acquired by Arrive (ParkMobile's parent) in 2025.	Prediction without sensors is commercially accepted.	https://business.parkopedia.com/parking-data [E]; https://business.parkopedia.com/about [E]
ParkMobile	US find/pay/reserve app.	Payment-centric, no enforcement.	https://parkmobile.io/ [E]
Dubai RTA / Parkin	Smart-scan cars with 24 cameras read plates in 360° at 80+ km/h with >99% claimed accuracy. **500+ AI curb/pole cameras plus 200 solar ANPR cameras (2026)** for occupancy and automatic payment. **Parkin–EMAAR (May 2026)** [F]: AI scans the plate, a missing permit is "flagged to a Parkin inspector at the Company's command centre.	**Exact match for your grace-period + officer-verification workflow.**	https://www.rta.ae/wps/portal/rta/ae/home/news-and-media/all-news/NewsDetails/starting-smart-screening-of-paid-public-parking-in-dubai [E]; https://www.thenationalnews.com/news/uae/2026/05/13/dubai-installs-ai-powered-cameras-to-detect-parking-offences-and-automate-payments/ [E]; https://feeds.dfm.ae/documents/2026/May/3/e5af9094-4097-4746-8a52-e4145e1bd997/EMAAR%20Malls%20Pod%20Enforcement%20ENG%20FINAL.Pdf.pdf [F]

System	Key facts	Takeaway	Source
	Following this review… a penalty notice may be issued. … a short grace period will be applied.”		
Singapore	Parking.sg (GovTech, since 2017) replaces paper coupons: 1.18M unique vehicles, cost $0.083 per session. HDB lets citizens report illegal parking via the **OneService app**, and a warden is alerted. Sun Singapore uses edge AI for plate reads, vacancy and violation detection.	Citizen reporting → warden dispatch is a norm.	https://products.open.gov.sg/parkingsg [E]; https://www.hdb.gov.sg/parking/parking-offences/report-a-parking-offence [E]; https://ir.amd.com/news-events/press-releases/detail/1205/… [E]
Barcelona ApparkB (2013–2019)	Pay by phone. From 2017 it gave colour-coded availability forecasts **built from payment data, not sensors**. Folded into the Smou app in 2019.	Forecasting from historical data alone is viable.	https://bsmsa.cat/en/press_releases/bsm-launches-new-updates-apparkb-and-areadum-applications [E]; https://1001smartcities.org/projects/barcelona-apparkb-parking/ [E]

Camera vs sensor: summary for judges

	Per-bay ground/IoT sensor	Camera / CV (incl. existing CCTV)
Coverage per device	1 bay	~4–8 bays (indoor) up to 100 (Cleverciti overhead)
Install	Trenching or drilling every bay	Pole mount; **zero new hardware if reusing police CCTV**
Weather/light	Robust	Glare, night and occlusion hurt accuracy
Failure mode	1 bay lost	All bays in view lost
Extra value	Occupancy only	Occupancy **plus** plates, stationary-vehicle detection, evidence
Cost (vendor-reported)	~$300–800 per sensor (Cocoparks table)	Camera systems ~$30–80 per space at scale (parkingtech.org review)

Sources: https://fleximodo.com/blog/2025/02/03/blog-parking-sensors-vs-cameras/ [E] (pro-sensor vendor); https://parkingtech.org/reviews/parking-occupancy-sensors-comparison/ [E]; https://

www.parking.net/parking-news/urbiotica/choosing-the-right-technology-for-smart-cities-iot-sensors-vs-cameras [E]; https://eleven-x.com/assets/Uploads/eleven-x-A-Closer-Look-at-Cameras-Vs-Sensors-Tech-Brief.pdf [E] (pro-sensor vendor). **All are vendor sources; present the figures as indicative.**

---

4. Similar hackathon projects

Smart India Hackathon

- **SIH 2023, PS SIH1515 (MoHUA): "Smart and Effective Real-time Management of Street Parking".** Listed in the SIH 2023 PS compilation. https://www.scribd.com/document/669583816/SIH-2023-Final [E]
  - **Winner: NMIMS Indore, "iPark"** (₹1 lakh prize; MoHUA said it would test it). Their own account [F] (https://www.linkedin.com/posts/sam-varghese2003_sih2023-hackathon-innovation-activity-7147620359205277697-oAlG):
    - **Approach:** used existing CCTV with CV to monitor areas; ML pushes real-time congestion data to officials; app to reserve spots; dynamic pricing; spreads demand across streets.
    - **USP:** "utilizing existing Government infrastructure".
    - **Problems they hit:** no-shows after reservations; "Indian cameras don't capture footages in high quality, hence… OCR won't work, especially at night"; tracking user location is a privacy and legal problem.
    - Press: https://curriculum-magazine.com/nmims-indore-students-craft-ipark-solution-for-city-parking-woes-in-the-smart-india-hackathon-2023/ [E]; https://mediaexpress24.com/technology/nmims-stme-students-craft-digital-solution-for-city-parking-woes/ [E]
  - Other SIH1515 teams: YOLOv5/v8 with ROI/IoU on parking videos; struggled with concurrency, edge vs cloud, and authenticating parked cars (QR vs ANPR). https://www.linkedin.com/posts/asheshjyoti_computervision-yolov8-opencv-activity-7141149510096896001-0eBv [E]. FASTag + ultrasonic prototype: https://www.linkedin.com/posts/akshatyadav19_smartindiahackathon-iot-smartparking-activity-7417862692805468160-MuSO [E]
- **SIH 2025, PS SIH25199 "Smart Parking Finder"** (idea deck): AI and IoT slot detection, app/web booking, prediction, dynamic pricing, EV filter. https://www.scribd.com/document/926991613/SIH2025-IDEA-PresentationNew [E]
- SIH 2024/2025/2026 PS portals: https://sih.gov.in/sih2024PS, https://www.sih.gov.in/sih2025PS, https://www.sih.gov.in/sih2026PS [E]. **I did not verify whether SIH 2024 had a parking or illegal-parking PS** (portals are JS-rendered). The SIH 2026 list has 233 PS (community mirror https://sih2026.vuce.in/ [E]) and could be searched for "parking".

Devpost / GitHub

Project	Approach	Source
UnPark	YOLO + DeepSORT tracking, urgency score (time × location × vehicle type), ANPR-ready OCR, evidence clips in MongoDB, NLP alert summaries. Built around Indian roadside parking.	https://devpost.com/software/unpark-fjwrd4 [E]
ParkGuard (HackSMU 2025)	YOLOv11 detects ADA symbols; a multimodal model flags obstructed or illegal parking.	https://devpost.com/software/hacksmu-2025-the-goat [E]
Gotcha! (UVA)	Drones plus YOLOv5 and plate OCR to scan lots for unregistered cars.	https://devpost.com/software/gotcha-2l13am [E]
Lot Vision		https://devpost.com/software/lot-vision [E]

Project	Approach	Source
	Raspberry Pi 5 + PaddleOCR ANPR checks plates against a permit DB; FastAPI + React dashboard.	
parkIn (hackCBS 5)	YOLOv4-tiny on CCTV feeds; SMS/email alerts; fines web app.	https://github.com/shouryade/parkIn [E]
Dard1ka/Ai-Ilegal-Parking	YOLOv8 + BotSORT, multi-zone ROI polygons, **3-stage Monitoring → Warning (60 s) → Violation (300 s)**, safety-triangle exception for breakdowns.	https://github.com/Dard1ka/Ai-Ilegal-Parking [E]
ParkIQ (BharatBytes Gridlock, built for BTP)	LightGBM spatio-temporal hotspot forecasting + YOLOv8 stationary-vehicle detection.	https://github.com/Vedang-M/BharatBytes-Gridlock-Stage-2 [E]

Common winning pattern: YOLO detection + multi-object tracking + ROI polygon + dwell-time threshold + evidence snapshot + dashboard. **What almost none of them have:** a police control plane (restrictions), human verification as a first-class step, citizen-report dedup, or a legal-evidence packet format. That gap is your differentiator.

---

5. Research papers & datasets

5.1 Camera-based occupancy detection

- **PKLot** (de Almeida et al., ESWA 2015): ~695,899 bay images from 3 camera views in sunny, overcast and rainy weather. Same-view accuracy >99%; **cross-lot generalisation is the hard part.** https://dlnext.acm.org/doi/10.1016/j.eswa.2015.02.009 [E]; readme https://www.inf.ufpr.br/lesoliveira/download/pklot-readme.pdf [E]
- **CNRPark+EXT / mAlexNet** (Amato et al., ESWA 2017): ~150k patches from 164 spaces and 9 cameras, with occlusion, shadows and seasons. A tiny CNN runs on a Raspberry Pi smart camera covering ~50 spaces. Datasets at http://cnrpark.it/ [E]; paper https://www.sciencedirect.com/science/article/pii/S095741741630598X [E]
- Improved MobileNetV3 (2023): 98.01% average on combined CNRPark-EXT + PKLot. https://exa.ai/library/publication/vdtdsy1sw95 [E]
- Slot detection + ResNet34 classifier (arXiv 2308.08192). https://arxiv.org/pdf/2308.08192 [E]
- Deep-learning video parking measurement with a tracking filter, comparable to radar benchmarks (arXiv 1902.07401). https://ar5iv.labs.arxiv.org/html/1902.07401 [E]
- MetaPKLot (2026), a harmonised benchmark. https://link.springer.com/article/10.1007/s00521-026-12398-0 [E]

5.2 Illegal-parking / stationary-vehicle detection

- **i-LIDS “Parked Vehicle” benchmark (AVSS 2007)**: DSP-based detection of vehicles in prohibited areas; 1-D transformation method (Lee et al., IEEE TCSVT 2009); ObjectVideo stationary-target detection. https://www.computer.org/csdl/proceedings-article/avss/2007/04425320/12OmNqGA54i [E]; https://scispace.com/pdf/real-time-illegal-parking-detection-in-outdoor-environments-21qjmtpyzn.pdf [E]
- GMM + Kalman filter; no-parking ROI with a **60 s stationary threshold**. https://www.computer.org/csdl/proceedings-article/aiccsa/2017/3581a840/12OmNz2TCDM [E]
- **SSD-based real-time illegal parking detection, 99% accuracy at 25 FPS** (Xie et al., arXiv 1710.02546). https://arxiv.org/abs/1710.02546 [E]
- YOLOv3 + IoU tracking + OpenALPR end-to-end pipeline (Liu 2019). [E]

- **Polygon restricted zones + temporal persistence threshold on public camera feeds** (IJCA). https://www.ijcaonline.org/archives/volume186/number70/computer-vision-and-deep-learning-based-approach-for-violations-due-to-illegal-parking-detection [E]
- **NYC study (Arnav 2025, Land Use Policy/Cities)**: AI on public traffic-camera feeds generated 558 complaints from 21 cameras in 5 days. **In 52.15% of cases vehicles were still illegally parked after police "responded".** This makes the case for closed-loop verification. https://www.sciencedirect.com/science/article/abs/pii/S026427512500592X [E] (fetch blocked)
- Mobile/cruise detection from dashcams (Kyushu 2025; SJTU 2025; AGCS 2025: precision 0.87, recall 0.88). [E]

5.3 Occupancy forecasting

- Graph CNN + LSTM block-level prediction using meter, traffic and weather data on SFpark (arXiv 1901.06758). https://arxiv.org/html/1901.06758v5 [E]
- LSTM with weather on Melbourne sensor data (MDPI Appl Sci 2022). https://www.mdpi.com/2076-3417/12/21/11046 [E]
- Hybrid ARIMA + LSTM + BPNN (Melbourne, SFpark). https://pmc.ncbi.nlm.nih.gov/articles/PMC11784715/ [E]
- XGBoost/LightGBM cross-city (SFpark → Melbourne → Birmingham): **hour-of-day, duration and day-of-week are the top SHAP features.** https://exa.ai/library/publication/p3fp9p660sk [E]
- Random Forest best among classic models on Melbourne data (R^2 0.87). https://www.mdpi.com/2624-6511/5/4/64 [E]
- Survey: Parking Prediction in Smart Cities (IEEE T-ITS 2023). https://dl.acm.org/doi/10.1109/TITS.2023.3279024 [E]
- Open data for a demo: Melbourne on-street bay sensors https://data.melbourne.vic.gov.au/explore/dataset/on-street-parking-bay-sensors [E]
- **Recommendation:** gradient boosting (LightGBM) on hour, day-of-week, holiday/festival flag, restriction flag and lagged occupancy. Easy to explain to judges, and literature supports it.

5.4 Citizen crowdsourced violation reporting

- **Israel "Road Guards"** (NRSA + police, 2016–): 3,400 volunteers, 64,000 reports in 24 months; **>80% rejected**, 13.7% got a notice letter, 4.3% a citation. **Quantifies the verification load.** https://exa.ai/library/publication/40xq2bd3x03 [E]
- KAIST **Mobile Roadwatch**: Korea's "Looking for a Witness" got >500k reports in ~18 months. https://phys.org/news/2017-04-traffic-safety-crowdsourced-violation-app.html [E]
- Pakistan crowdsourced framework with two-stage review (spam filter, then expert): of 53 reports, 15 were spam, 35 went to experts and 23 were marked valid (exact wording on the page is ambiguous). https://www.scitepress.org/Papers/2018/66803/66803.pdf [E]
- ML-enhanced Public Eye app study (Bengaluru). https://exa.ai/library/publication/bj3pslzxd3h [E]
- Ahmedabad e-challan analysis of 3M+ challans: repeat offenders, hotspots, **festival-day spikes and dips**. Supports your festival restrictions and hotspot features. https://ar5iv.labs.arxiv.org/html/1909.08106 [E]
- Waze reports: 43.2% coverage of ATMS incidents, 9.8 min earlier. https://journals.sagepub.com/doi/10.1177/0361198118790619 [E]

5.5 Duplicate complaint merging / clustering

- **Spatio-temporal graph clustering of duplicate civic reports** (Urb-IoT 2014 and follow-up 2015). Parameter-free graph clustering beat DBSCAN-style methods on labelled duplicate data. https://dl.acm.org/doi/abs/10.4108/icst.urb-iot.2014.257282 [E]
- **Nature Computational Science 2023**: duplicate-report rates in NYC and Chicago 311 used to estimate under-reporting and spatial disparity. **Duplicates are signal (severity/priority), not just noise.** https://www.nature.com/articles/s43588-023-00572-6 [E]

- Complaint grouping with NLP + geo, comparing 6 models (IJERT 2026): **sentence embeddings + location most reliable.** https://www.ijert.org/complaint-matching-and-grouping-using-nlp-and-geo-spatial-techniques-a-comparative-study-ijertconv14is010001 [E]
- CivicConnect (SSRN 2026): semantic de-dup with geospatial and language models. [E]
- Example implementation: PostGIS 50 m radius dedup (City-Sync). https://github.com/princ3kr/City-Sync [E]
- **Recommended algorithm for the demo:** candidate duplicates = same **plate** (if OCR'd) **or** within 30–50 m **and** a ±30–60 min window **and** the same category. Then confirm with image perceptual-hash or CLIP similarity and text embeddings. Merge into one "incident" with a `report_count` that raises priority. Run DBSCAN/HDBSCAN on (lat, lon, scaled time) for hotspots.

---

6. Gaps / differentiators this project can claim

1. **Zero new field hardware.** Reuses traffic-police ITMS/ANPR/CCTV already being installed under Smart Cities and Rule 167A. Contrast: Chennai, NDMC and BMC sensor/contract models stalled; SFpark sensors were costly and short-lived. (Caveat: police CCTV angles may not cover all bays. Claim "coverage where cameras exist" and fill gaps with forecasting and citizen reports.)
2. **Police control plane for parking supply.** Officers can open, close or temporarily restrict zones (festival, VIP, roadwork) and the change reflects instantly on the citizen map. No Indian system found does this. ASTraM's event simulation and NDMC's "special events" tariffs come closest.
3. **Human-in-the-loop enforcement by design.** The grace period, then an AI flag, then **officer verification**, then a challan-ready packet. This matches MV Act s.127 (a uniformed officer authorises towing), Rule 167A evidence requirements and Dubai Parkin's 2026 model. It also answers the Kerala lesson (1.04 lakh challans from 20.4 lakh detections) and the NYC lesson (52% unresolved after police response).
4. **One pipeline for camera flags and citizen reports.** Citizen reports are deduplicated, clustered and cross-checked against CCTV before reaching an officer. Prahari, Citizen Sentinel and ASTraM forward raw reports.
5. **Duplicates become priority signals and hotspots** (backed by the Nature Comp Sci 2023 finding), feeding patrol planning and the forecasting model.
6. **eChallan-ready, not eChallan-dependent.** The evidence packet mirrors eChallan's challan fields (lat-long, time, offence section, photo, 65B metadata) for later NAPIX integration by the state.
7. **Privacy by design.** Edge or on-prem inference; the public map shows only counts; plates and faces are blurred except for officers; role-based access and retention aligned to Rule 167A(9); citizen PII minimised. Grounded in DPDP s.8(5) and s.17.
8. **Fits ICCC maturity scoring.** It adds a "City Parking Solutions" and "Traffic enforcement via ITMS" use case to IMAF 2.0. Cities are already told to make their ICCCs sustainable.

---

7. Likely judge questions, with suggested answers

Q1. How much does it cost? No new cameras or sensors. Costs are software, GPU/edge inference and officer time. Compare per-bay sensors at ~$300–800 each (vendor table) or failed multi-crore PPP tenders (BMC, Chennai). Kerala's AI cameras cost ₹9.40 cr against ₹59.46 cr in fines, though that is helmet/seatbelt enforcement, not parking. One edge GPU box can serve several feeds. Give your own measured FPS per stream from the demo.

Q2. How does it scale? Each camera is processed independently (stateless detector plus per-camera tracker), so it scales horizontally. Run it at the ICCC or edge and send only events and counts upstream. That is the same pattern as the Bhopal parking-command-centre-to-ICCC integration. Cleverciti shows one overhead view can cover up to 100 bays.

Q3. Is it accurate? What about night, rain and occlusion? Literature: >99% same-view occupancy accuracy (PKLot); 98% on CNRPark+PKLot (MobileNetV3); 99% / 25 FPS for SSD illegal-parking detection. Cross-camera generalisation is the known weak point, so we fine-tune per camera and calibrate ROI polygons. Indian CCTV OCR is weak at night (the SIH1515 winners said so), **so plate OCR is optional and the officer confirms.** Report your own precision and recall on a held-out clip.

Q4. False positives: a car stopped at a signal, a breakdown, a drop-off? Multi-stage logic: ROI polygon (only no-parking zones), dwell time beyond a configurable grace period (e.g. 60 s warning, 5 min violation, as in prior work), tracker ID persistence to tell a stopped car from a parked one, exceptions (hazard lights, warning triangle, emergency vehicles, police-whitelisted zones). **Nothing is challaned without an officer.**

Q5. Is a camera-based parking challan legally valid? Yes. Rule 167A explicitly lists s.122 parking at an unauthorised location. Conditions: device approval certificate renewed yearly; photo with plate, date/time/place and the section violated; 65B certificate (BSA 2023 s.63 equivalent); notice within 15 days; records kept until disposal. Towing or clamping under s.127 needs a uniformed officer's authorisation. Our officer-verification step covers both.

Q6. Privacy and DPDP? Enforcement processing falls under s.17(1)(c), but s.8(5) security safeguards still apply. Aggregated analytics fall under s.17(2)(b). Citizen complainant data needs notice and minimisation. The public map shows no video, plates or faces. Access control, audit logs and data kept in India mirror MoRTH's NTR data-sharing policy.

Q7. How will you integrate with eChallan without access? In the prototype, we generate a challan-ready packet in eChallan's documented field schema. In production, the state traffic police applies to MoRTH for NAPIX API access (CERT-In audit, IP whitelisting, yearly renewal), or uses the existing ITMS Challan module that NIC already offers to smart-city ITMS projects.

Q8. Won't citizens spam or abuse the reporting? Israel's Road Guards rejected >80% of reports. We dedupe and cluster, require GPS and timestamp captured in-app (gallery uploads disallowed, as Prahari does), score reporter reputation, cross-check against CCTV, and use rate limits. The officer still decides. We don't pay cash per report, which is one of Prahari's spam risks.

Q9. Why would police adopt this when Prahari and ASTraM exist? We complement them: we add parking availability, a restriction control plane and pre-verification dedup. ASTraM has APIs and map engines, so it can consume our events. Officer time saved through merged duplicates and CCTV corroboration is the adoption metric.

Q10. What if a camera doesn't cover a street? We show "unknown" honestly rather than guessing. We fill gaps with a forecast based on historical and time features (Barcelona and Parkopedia predict without sensors), plus citizen reports.

Q11. Forecasting: how and how good? LightGBM on hour, day-of-week, holiday/festival flag, active restrictions and lagged occupancy. SHAP shows hour, duration and day-of-week dominate (cross-city study). Report MAE at 15, 30 and 60 min horizons on your own data or Melbourne open data.

Q12. What's novel versus the SIH 2023 winner iPark? iPark covered availability, reservation and pricing. Ours adds police open/close/restrict controls, an enforcement workflow with grace period and officer verification, citizen reporting with dedup, hotspot maps and an eChallan evidence path. We drop reservations of public street space, which iPark itself found was abused by no-shows.

Q13. What about pilot location and sustainability? Puducherry is installing ANPR/RLVD smart signals and a RailTel ICCC whose scope includes on-street parking management. Chennai is re-tendering smart parking for 2026. The software-only model avoids the contractor collapse seen in Chennai.

---

8. Corrections to make in the one-pager

- “Delhi Traffic Police Traffic Sentinel” → **“Traffic Prahari (formerly Traffic Sentinel, relaunched 1 Sep 2024)”**.
- “BTP Public Eye” → **discontinued Mar 2025; now part of BTP ASTraM**.
- “MoHUA TMICC guidance”: the document was issued under **MoUD** (MoHUA’s former name) through GEF-SUTP and is hosted on NIUA SmartNet. Fine to call it MoHUA guidance, but pair it with **IMAF 2.0 (ICCC Maturity Assessment Framework)**, which is current.
- Add the legal hook: **MV Act s.136A + CMVR Rule 167A (2021)** and **s.122/127/177**.
- Add **mParivahan Citizen Sentinel** as the national citizen-reporting precedent.

9. Not covered or unverified

- MyParkings (India): no source found.
- Whether SIH 2022 or 2024 had a specific illegal-parking problem statement: not verified (portal is JS-rendered).
- The TMICC document’s full contents: page fetch failed; only the description excerpt was seen.
- Most items marked `[E]` are from search excerpts. Open the URL before putting a number on a slide. Fully read `[F]`: eChallan NIC brochure, MoRTH NTR data-sharing policy, PIB Rule 167A, MV Act s.127, DPDP s.17, BTP ASTraM page, Traffic Prahari page, NDMC smart parking page, Chennai 1001smartcities entry, Kerala AI-camera India Today article, Parkin–EMAAR release, SIH1515 winner’s LinkedIn post.

Part 3. Stack, data model, algorithms, APIs, demo, build plan

Researched 2026-10-11. Built from the one-page brief (*files/extracted/Smart_Parking_Hackathon_One_Page-1/content.md*). Every URL below was opened or showed up in a search result during this research. Items marked “(search only)” were seen in search results but not opened.

---

0. TL;DR (the decisions)

Layer	Pick	Why (short)
CV worker	Python + Ultralytics YOLO (nano model, CPU) + built-in ByteTrack + Roboflow `supervision` PolygonZone	Runs on a laptop with no GPU. Polygon zones and dwell time come ready-made.
Camera simulation	MediaMTX + `ffmpeg -re -stream_loop -1 ... -f rtsp`	Turns MP4s into “live CCTV” RTSP feeds. One binary, MIT licence.
Backend	**FastAPI** (Python)	Same language as the CV worker. Async WebSockets, Pydantic validation and Swagger docs come free, which makes it a good demo prop.
DB	**PostgreSQL 16 + PostGIS** (Docker locally; Neon free tier in the cloud)	Real polygon and distance queries (`ST_DWithin`, `ST_Within`, `ST_ClusterDBSCAN`).
Realtime	**FastAPI native WebSockets** (one `/ws` channel, typed events). Fallback: SSE / polling	One process, no extra vendor, works offline.
Maps	**Leaflet + react-leaflet + OSM tiles**, `leaflet.heat`, `h3-js`	Free, no API key, and the plugin ecosystem covers heatmaps and hex bins.
Navigation	**Google Maps URL deep link** (no key) + optional OSRM route preview	Opens turn-by-turn on the user’s own phone. `travelmode=two-wheeler` fits India.
Frontend	**React + Vite + TypeScript + Tailwind + shadcn/ui**, two routes: `/` (citizen) and `/police`	Matches the builder’s existing React/Vercel skills. Vite makes the PWA easy.
PWA	`vite-plugin-pwa`	Camera + GPS reporting, installable, offline shell.
Hosting	Frontend → Vercel. API → Render free (or run on the laptop). DB → Neon free	₹0. **Demo runs from the laptop**, with the cloud as backup.

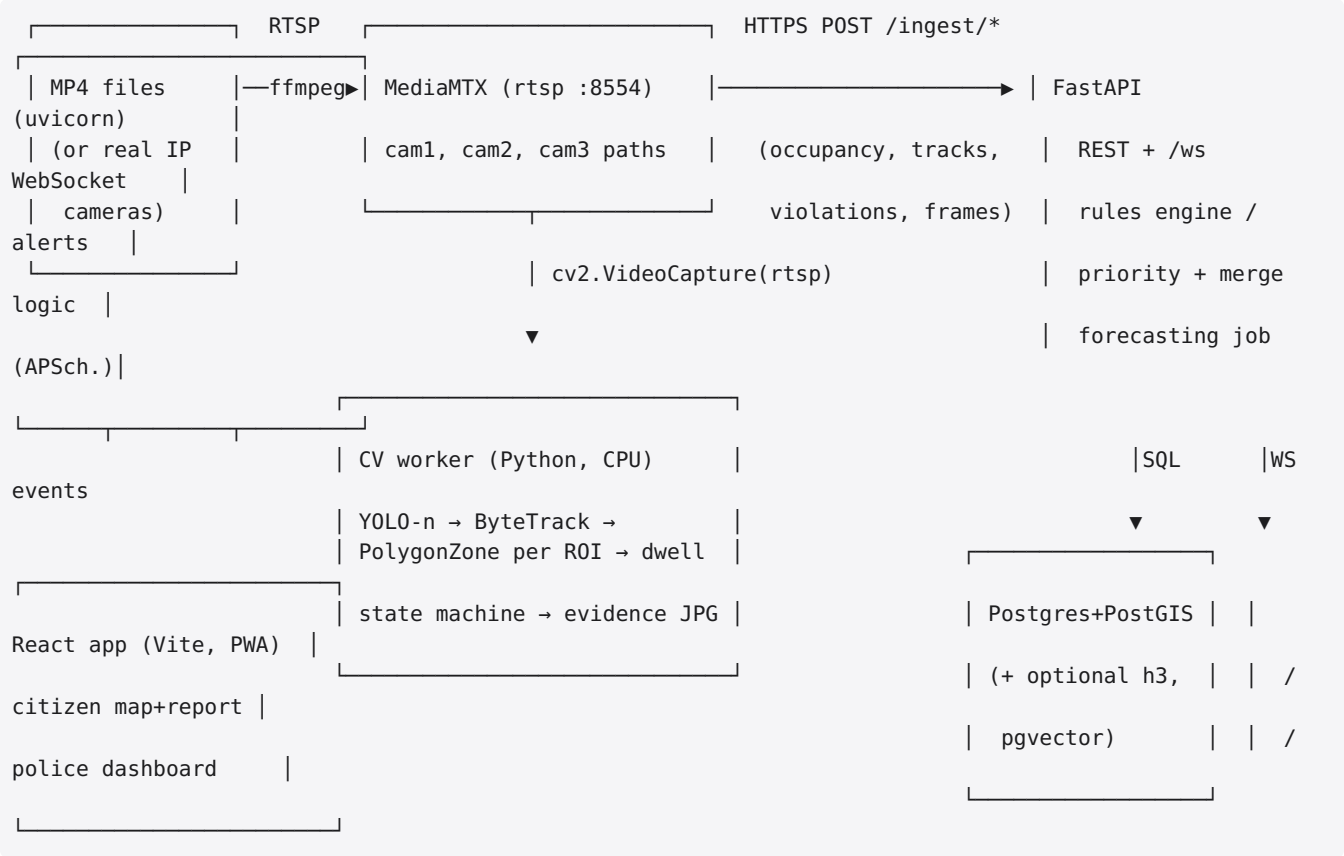
Why not Supabase as the core? It’s a fine alternative: PostGIS and Realtime are built in, the free tier allows 200 concurrent realtime connections, and it saves writing auth. But free projects pause after a week of low activity and you only get two active projects. The bigger issue is that you’d still need a Python service for CV and alert logic. One FastAPI process that owns state, alerts and WebSockets is simpler to reason about and to demo offline. Use Supabase only if the team wants auth and storage handed to them. (Supabase pricing, project pausing, Realtime limits)

Why not Node? Node with Socket.IO would be just as good for realtime. But the CV, forecasting (pandas/LightGBM), clustering (scikit-learn) and image hashing are all Python. One language means one data model and one set of Pydantic schemas shared by the worker and the API.

---

1. Architecture & stack

1.1 Component diagram



Key design rule: the CV worker is dumb and replaceable. It only reports facts: counts per zone, tracks in ROIs, and evidence snapshots. **All decisions** (status, grace periods, merging, priority, alerts) live in the backend. That way the “Simulate vehicle” button and the real CV path go through the same code, and the demo cannot drift from the real logic.

1.2 Backend: FastAPI

- Native WebSockets with a `ConnectionManager.broadcast()` pattern straight from the docs. FastAPI warns that the in-memory list only works for a single process, and points to `encode/broadcaster` (Redis/Postgres) for multiple workers. **Run one uvicorn worker.** That’s plenty for a hackathon. (FastAPI WebSockets)
- Libraries: `sqlalchemy` 2 + `geoalchemy2` (or `asyncpg` + raw SQL), `pydantic`, `apscheduler` for periodic jobs (forecast refresh, dwell sweep, priority re-score), `pillow` + `imagehash`, `shapely`, `h3`.
- Swagger UI at `/docs`. Show it for 5 seconds in the pitch as proof that real APIs exist.

1.3 Database: PostgreSQL + PostGIS

- `geography(Polygon,4326)` for zones and `geography(Point,4326)` for complaints, so `ST_DWithin(a,b,50)` works in metres with no projection fuss.
- For DBSCAN in metres, transform to UTM 44N (EPSG:32644). Karaikal (≈79.8°E) and Puducherry fall in zone 44, which covers 78–84°E.
- **Local:** `docker run postgis/postgis:16-3.4`.
- **Cloud:** Neon’s free plan supports PostGIS and has no time limit. It gives 0.5 GB per project and 100 CU-hours a month, and scales to zero after 5 minutes idle, waking in a few hundred ms. (Neon PostGIS, Neon free tier) Neon also lists `h3_postgis` and `pgrouting` as available extensions. (Neon extensions)

- Avoid Render's free Postgres: it **expires 30 days after creation** and has no backups. (Render free)
- Supabase docs: install PostGIS into an `extensions` schema rather than `public`. (Supabase PostGIS)

1.4 Realtime push: options compared

Option	Pros	Cons	Verdict
FastAPI WebSocket	No vendor, offline, bidirectional, one codebase	Single process (fine)	**Primary**
SSE (`text/event-stream`)	Simpler, auto-reconnect, works through proxies	Server→client only	Good fallback for the citizen map
Supabase Realtime (Postgres Changes)	Zero server code, DB change → client	Needs Supabase as DB. Docs call Broadcast the scalable choice and Postgres Changes the simpler one	Alternative if you go all-in on Supabase (docs)
Socket.IO	Rooms, reconnection	Needs `python-socketio`, protocol overhead	Not needed

Client: one reconnecting WebSocket hook in React. On reconnect, call `GET /zones` to resync, so a missed event can never leave a stale map on stage.

1.5 Maps

- **Leaflet + react-leaflet + OSM raster tiles**: the simplest path. Draw zone polygons coloured by status. Use `CircleMarker`s for reports and a pulsing icon for incidents.
- **Heatmap:** `leaflet.heat` (`L.heatLayer([[lat,lng,intensity],...],{radius:25})`, BSD-2). (GitHub) A React wrapper `react-leaflet-heat-layer` targets react-leaflet v4 (npm.io), or write a 15-line `useMap()` hook that adds `L.heatLayer` yourself. That's more reliable.
- **Hex bins:** `h3-js` `latLngToCell` + `cellsToMultiPolygon` → GeoJSON. (h3-js)
- **MapLibre GL** is the vector-tile alternative: smoother and 3D-capable, but more setup and needs a tile source. Skip it for a 48-hour build.
- **Navigation deep link (no API key):** `https://www.google.com/maps/dir/?api=1&destination=<lat>,<lng>&travelmode=driving|two-wheeler&dir_action=navigate`. `api=1` is required. Omit `origin` and it uses the device location. (Google Maps URLs, two-wheeler mode) There's a known iOS quirk where `travelmode` can cause a first-launch "Unsupported link", so test on the demo phone. (SO)
- **In-app route preview (optional):** OSRM `GET /route/v1/driving/{lng,lat;lng,lat}?overview=full&geometries=geojson`. The public demo server is for **non-commercial use, max 1 request/second, no uptime guarantee**, so cache the result or precompute for the demo. (OSRM HTTP API, demo server policy)

1.6 Frontend

- React + Vite + TS + Tailwind + shadcn/ui (Card, Badge, Table, Sheet, Dialog, Toast/Sonner). TanStack Query for REST, Zustand for live zone state from WS events.
- Citizen (`/`): mobile-first map, bottom sheet with nearby zones sorted by distance, plus a "Report" FAB.
- Police (`/police`): desktop 3-pane layout. Left: KPI tiles + alert feed. Centre: map. Right: priority queue / incident detail with CCTV snapshot, merged reports, and Verify/Dismiss buttons.
- Next.js is unnecessary (no SSR need), and it complicates WebSockets on Vercel.

1.7 PWA for citizen reporting

- `vite-plugin-pwa`: zero-config manifest + Workbox service worker, with a React hook `useRegisterSW` via `virtual:pwa-register/react`. (docs, React)

- Capture: `<input type="file" accept="image/*" capture="environment">` (most reliable on Android Chrome) + `navigator.geolocation.getCurrentPosition({enableHighAccuracy:true})`. Compress client-side to ≤1280px JPEG before upload. Strip EXIF but keep the GPS from the browser.
- Offline queue (stretch): store pending reports in IndexedDB and flush when back online.
- Geolocation and camera need **HTTPS** (or localhost). The Vercel URL gives you that. For the LAN demo, use the Vercel frontend pointed at the laptop API through a tunnel, or `vite --https` with mkcert.

1.8 Video ingest

- **Simulate CCTV:** `./mediamtx` then `ffmpeg -re -stream_loop -1 -i beach_road.mp4 -c copy -f rtsp rtsp://localhost:8554/cam1`. `-re` paces playback in real time and `-stream_loop -1` loops forever. (MediaMTX basic usage, publish with FFmpeg) MediaMTX can also auto-start the ffmpeg loop with `runOnDemand` in `mediamtx.yml`. (on-demand publishing) Repo: bluenviron/mediamtx (MIT, ~20k stars).
- If the MP4 has odd codecs or B-frames, re-encode once to H.264 baseline/`-tune zerolatency`, then stream with `-c copy`. Pass `-rtsp_transport tcp` when reading to avoid UDP packet loss. (RTSP features)
- **Read:** `cv2.VideoCapture("rtsp://localhost:8554/cam1", cv2.CAP_FFMPEG)` in a dedicated thread that keeps only the latest frame (drop stale frames). Run inference at **2–5 fps**, which is plenty for parking.
- **Detection/tracking:** `YOLO("yolo26n.pt")` or `yolo11n.pt`, `model.track(frame, persist=True, tracker="bytetrack.yaml", classes=[2,3,5,7], imgsz=640)` (COCO car/motorcycle/bus/truck). Use `persist=True` only for consecutive frames of the same stream. (Ultralytics track mode) Export to ONNX/OpenVINO for faster CPU inference if needed.
- **Zones and dwell:** `supervision.PolygonZone(polygon, triggering_anchors=(sv.Position.BOTTOM_CENTER,))` + `zone.trigger(detections)`. The `time_in_zone` example already has per-track timers and a `draw_zones.py` polygon drawing tool. (supervision count-in-zone, time_in_zone example)
- **Parking-slot-level option:** Ultralytics `solutions.ParkingManagement(model=..., json_file="bounding_boxes.json")` + the `ParkingPtsSelection()` GUI annotator. On Fedora it needs `sudo dnf install python3-tkinter`. (guide)
- **Licence note:** Ultralytics is **AGPL-3.0** (repo). That's fine for a hackathon or open-source repo. Mention in Q&A that production would need an enterprise licence or a permissive detector such as RF-DETR.
- **Test footage:** use a CC/YouTube street-parking clip filmed from above, or record a 2-minute phone video of a real Karaikal street from a rooftop. Static datasets for validating counts: CNRPark+EXT, ~150k labelled patches from 9 cameras (cnrpark.it), and PKLot, 12,417 images at 1280×720 (UFPR). These are stills, so build a slideshow MP4 with ffmpeg if needed.

1.9 Free hosting

Piece	Where	Free-tier gotchas
Frontend	Vercel (Hobby)	Fine. Vercel now has WebSockets on Functions (public beta, Fluid compute) (changelog), but a long-running FastAPI process belongs on a real server.
API	Render free web service	Spins down after **15 min** without HTTP requests or WebSocket messages, and takes ~1 min to wake. 750 instance-hours/month, 0.1 CPU / 512 MB. (Render free, WS keeps alive) **Hit the URL 5 min before judging.**
DB	Neon free	Scale-to-zero after 5 min, sub-second resume. (Neon)
CV worker	**Laptop** (always)	

Piece	Where	Free-tier gotchas
		Free cloud CPUs (0.1 CPU) can't run YOLO. The worker POSTs results to the API, local or cloud.
Images	Local disk `/media` served by FastAPI. Cloud: Supabase Storage free 1 GB or Cloudflare R2	Render's free disk is wiped on spin-down, so don't store evidence there long-term.

Primary demo mode = everything on the laptop (`docker compose up`: postgis + api + mediamtx + worker + vite). The phone joins over the laptop hotspot. The cloud deploy is the judges' "try it yourself" link and the fallback.

---

2. Data model (PostgreSQL + PostGIS)

```
CREATE EXTENSION IF NOT EXISTS postgis;
-- optional: CREATE EXTENSION IF NOT EXISTS vector;   -- CLIP embeddings (pgvector)
-- optional: CREATE EXTENSION IF NOT EXISTS h3; CREATE EXTENSION IF NOT EXISTS h3_postgis;

CREATE TYPE zone_kind      AS ENUM ('authorised_parking','no_parking','no_stopping','loading','emerge
        ncy_access');
CREATE TYPE zone_status    AS ENUM ('available','almost_full','full','closed','restricted','unknown');
CREATE TYPE incident_type  AS ENUM ('illegal_parking','double_parking','blocked_entrance','blocked_fo
        otpath',
                                    'no_parking_violation','overflow','spillover','other');
CREATE TYPE incident_state AS ENUM
        ('open','assigned','verified','dismissed','resolved','challan_sent');
CREATE TYPE officer_role   AS ENUM ('officer','supervisor','admin');

-- ── Zones ──────────────────────────────────────────────────────────────
CREATE TABLE zones (
  id              SERIAL PRIMARY KEY,
  code            TEXT UNIQUE NOT NULL,              -- 'KKL-BEACH-01'
  name            TEXT NOT NULL,                     -- 'Beach Road (Karaikal) — North stretch'
  kind            zone_kind NOT NULL DEFAULT 'authorised_parking',
  boundary        GEOGRAPHY(POLYGON,4326) NOT NULL,
  centroid        GEOGRAPHY(POINT,4326) GENERATED ALWAYS AS (ST_Centroid(boundary::geometry)::geograp
        hy) STORED,
  buffer_m        INT NOT NULL DEFAULT 25,           -- spillover ring width
  capacity_car    INT NOT NULL DEFAULT 0,
  capacity_2w     INT NOT NULL DEFAULT 0,            -- two-wheelers matter in India
  pcu_weight_2w   REAL NOT NULL DEFAULT 0.25,        -- 4 bikes ≈ 1 car slot if mixed
  vehicle_types   TEXT[] DEFAULT '{car,2w}',
  fee_info        TEXT,                              -- 'Free' / '₹10/hr'
  status          zone_status NOT NULL DEFAULT 'unknown',   -- derived (cached)
  manual_override zone_status,                       -- police: closed/restricted/null
  override_reason TEXT,                              -- 'Festival', 'VIP movement', 'Road work'
  override_until  TIMESTAMPTZ,
  occupied_now    INT NOT NULL DEFAULT 0,
  occ_updated_at  TIMESTAMPTZ,
  near_emergency  BOOLEAN NOT NULL DEFAULT false,    -- precomputed from OSM (hospital/fire station
        within 100 m)
  created_at      TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX zones_boundary_gix ON zones USING GIST (boundary);

-- Opening hours & recurring rules (e.g., 'no parking 8-11 AM Sun market')
CREATE TABLE zone_schedules (
  id        SERIAL PRIMARY KEY,
  zone_id   INT REFERENCES zones(id) ON DELETE CASCADE,
  dow       SMALLINT[] NOT NULL,                     -- {0..6}, 0=Mon
  opens     TIME NOT NULL,
  closes    TIME NOT NULL,                           -- handle overnight with closes < opens
```

```
  rule      TEXT NOT NULL DEFAULT 'open'             -- 'open' | 'closed' | 'restricted_2w_only' ...
);
-- One-off scheduled closures (festival, VIP) — also drives "closes in 30 min"
CREATE TABLE zone_closures (
  id         SERIAL PRIMARY KEY,
  zone_id    INT REFERENCES zones(id) ON DELETE CASCADE,
  starts_at  TIMESTAMPTZ NOT NULL,
  ends_at    TIMESTAMPTZ,
  kind       zone_status NOT NULL DEFAULT 'closed',
  reason     TEXT,
  created_by INT,
  grace_min  INT NOT NULL DEFAULT 15               -- time vehicles get to leave after closure
);

-- ── Cameras ──────────────────────────────────────────────────────────
CREATE TABLE cameras (
  id            SERIAL PRIMARY KEY,
  code          TEXT UNIQUE NOT NULL,              -- 'CAM-KKL-07'
  rtsp_url      TEXT NOT NULL,                     -- rtsp://localhost:8554/cam1
  location      GEOGRAPHY(POINT,4326),
  coverage      GEOGRAPHY(POLYGON,4326),           -- ground footprint (for evidence matching)
  frame_w       INT, frame_h INT,
  homography    DOUBLE PRECISION[9],               -- optional pixel→lat/lng (cv2.findHomography)
  last_seen_at  TIMESTAMPTZ,                       -- heartbeat → 'camera offline' alert
  fps_target    REAL DEFAULT 3,
  active        BOOLEAN DEFAULT true
);
CREATE INDEX cameras_coverage_gix ON cameras USING GIST (coverage);

-- ROI polygons in PIXEL space, each mapped to a zone
CREATE TABLE camera_rois (
  id         SERIAL PRIMARY KEY,
  camera_id  INT REFERENCES cameras(id) ON DELETE CASCADE,
  zone_id    INT REFERENCES zones(id) ON DELETE CASCADE,
  role       TEXT NOT NULL CHECK (role IN ('count','no_parking','spillover_ring','entrance_keepclear')
        ),
  polygon_px JSONB NOT NULL,                       -- [[x,y],...] in the camera frame
  weight     REAL NOT NULL DEFAULT 1.0             -- if one zone spans several cameras
);

-- ── Time series ──────────────────────────────────────────────────────
CREATE TABLE occupancy_snapshots (
  zone_id     INT REFERENCES zones(id) ON DELETE CASCADE,
  ts          TIMESTAMPTZ NOT NULL,
  occupied    INT NOT NULL,
  occupied_2w INT DEFAULT 0,
  capacity    INT NOT NULL,                        -- effective capacity at that time
  source      TEXT NOT NULL DEFAULT 'cctv',        -- 'cctv' | 'manual' | 'simulated' | 'seed'
  confidence  REAL,
  PRIMARY KEY (zone_id, ts)
);
CREATE INDEX occ_ts_idx ON occupancy_snapshots (ts);
-- Pre-aggregated profile for "usually full 6–9 PM" + baseline forecast
CREATE MATERIALIZED VIEW occupancy_profile AS
SELECT zone_id,
       EXTRACT(ISODOW FROM ts AT TIME ZONE 'Asia/Kolkata')::int - 1 AS dow,
       (EXTRACT(HOUR FROM ts AT TIME ZONE 'Asia/Kolkata')*4
        + FLOOR(EXTRACT(MINUTE FROM ts AT TIME ZONE 'Asia/Kolkata')/15))::int AS slot15,
       AVG(occupied::real / NULLIF(capacity,0)) AS mean_ratio,
       PERCENTILE_CONT(0.8) WITHIN GROUP (ORDER BY occupied::real/NULLIF(capacity,0)) AS p80_ratio,
       COUNT(*) AS n
FROM occupancy_snapshots
WHERE ts > now() - INTERVAL '28 days'
GROUP BY 1,2,3;
```

```sql
-- ── Tracks / detections (summarised, not per-frame) ──────────────────
CREATE TABLE tracks (
  id              BIGSERIAL PRIMARY KEY,
  camera_id       INT REFERENCES cameras(id),
  tracker_id      INT NOT NULL,                    -- ByteTrack id (unique per camera session)
  session_id      TEXT NOT NULL,                   -- worker restart → new session
  vehicle_class   TEXT,                            -- car / motorcycle / bus / truck
  roi_id          INT REFERENCES camera_rois(id),
  zone_id         INT REFERENCES zones(id),
  first_seen      TIMESTAMPTZ NOT NULL,
  stationary_since TIMESTAMPTZ,
  last_seen       TIMESTAMPTZ NOT NULL,
  last_bbox       REAL[4],                         -- x1,y1,x2,y2 (px)
  state           TEXT NOT NULL DEFAULT 'moving',  -- moving|stationary|in_grace|flagged|gone
  snapshot_url    TEXT,
  UNIQUE (camera_id, session_id, tracker_id)
);
CREATE INDEX tracks_zone_state_idx ON tracks (zone_id, state);

-- ── Incidents (the merged "case"), complaints, evidence ──────────────
CREATE TABLE incidents (
  id              BIGSERIAL PRIMARY KEY,
  type            incident_type NOT NULL,
  state           incident_state NOT NULL DEFAULT 'open',
  location        GEOGRAPHY(POINT,4326) NOT NULL,
  zone_id         INT REFERENCES zones(id),
  severity        SMALLINT NOT NULL DEFAULT 2,     -- 1..5
  blocks_emergency BOOLEAN NOT NULL DEFAULT false,
  report_count    INT NOT NULL DEFAULT 0,          -- distinct reporters
  cctv_status     TEXT NOT NULL DEFAULT 'none',    -- none|possible|confirmed
  priority        REAL NOT NULL DEFAULT 0,         -- 0..100, recomputed
  first_seen      TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_activity   TIMESTAMPTZ NOT NULL DEFAULT now(),
  assigned_to     INT,
  verified_by     INT,
  verified_at     TIMESTAMPTZ,
  vehicle_plate   TEXT,                            -- officer-entered only (no ANPR in MVP)
  echallan_ref    TEXT,                            -- future hook
  h3_r9           TEXT                             -- for hotspot aggregation
);
CREATE INDEX incidents_loc_gix ON incidents USING GIST (location);
CREATE INDEX incidents_open_idx ON incidents (state, priority DESC);

CREATE TABLE complaints (
  id            BIGSERIAL PRIMARY KEY,
  incident_id   BIGINT REFERENCES incidents(id),
  category      incident_type NOT NULL,
  description   TEXT,
  location      GEOGRAPHY(POINT,4326) NOT NULL,
  gps_accuracy_m REAL,
  photo_url     TEXT,
  photo_phash   BIT(64),                           -- imagehash.phash → 64-bit
  -- photo_embedding VECTOR(512),                  -- optional CLIP
  reporter_hash TEXT,                              -- hashed phone/device id (privacy)
  merge_score   REAL,                              -- why it was merged (explainability)
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX complaints_loc_gix ON complaints USING GIST (location);

CREATE TABLE incident_evidence (
  id          BIGSERIAL PRIMARY KEY,
  incident_id BIGINT REFERENCES incidents(id) ON DELETE CASCADE,
```

```
  kind        TEXT NOT NULL,                        -- cctv_snapshot|cctv_clip|citizen_photo|
        officer_photo
  url         TEXT NOT NULL,
  camera_id   INT REFERENCES cameras(id),
  track_id    BIGINT REFERENCES tracks(id),
  captured_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE alerts (
  id          BIGSERIAL PRIMARY KEY,
  kind        TEXT NOT NULL,                        -- almost_full|overflow|possible_illegal|
        spillover|camera_offline|closing_soon
  zone_id     INT REFERENCES zones(id),
  incident_id BIGINT REFERENCES incidents(id),
  message     TEXT NOT NULL,
  level       TEXT NOT NULL DEFAULT 'info',         -- info|warn|critical
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  acked_by    INT, acked_at TIMESTAMPTZ,
  dedupe_key  TEXT UNIQUE                           -- e.g. 'almost_full:12:2026-10-11T18'
);

-- ── People & audit ──────────────────────────────────────────────────────
CREATE TABLE officers (
  id          SERIAL PRIMARY KEY,
  badge_no    TEXT UNIQUE NOT NULL,
  name        TEXT NOT NULL,
  role        officer_role NOT NULL DEFAULT 'officer',
  station     TEXT,
  pass_hash   TEXT NOT NULL,
  last_location GEOGRAPHY(POINT,4326)
);

CREATE TABLE audit_log (
  id          BIGSERIAL PRIMARY KEY,
  actor_type TEXT NOT NULL,                         -- officer|system|citizen|cv_worker
  actor_id    TEXT,
  action      TEXT NOT NULL,                        -- zone.close, incident.verify,
        complaint.merge ...
  entity      TEXT NOT NULL,                        -- 'zone:12'
  before      JSONB, after JSONB,
  reason      TEXT,
  at          TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Useful queries:

```
-- Nearby open zones for the citizen (KNN, uses GiST index)
SELECT id, name, status, capacity_car - occupied_now AS free,
       ST_Distance(centroid, ST_MakePoint(:lng,:lat)::geography) AS dist_m
FROM zones WHERE kind='authorised_parking'
ORDER BY centroid <-> ST_MakePoint(:lng,:lat)::geography LIMIT 10;

-- Which camera can corroborate a complaint?
SELECT c.id FROM cameras c
WHERE c.active AND ST_Intersects(c.coverage, ST_MakePoint(:lng,:lat)::geography);

-- Is a reported point inside a zone, or just outside it (spillover)?
SELECT id, ST_Covers(boundary, p) AS inside,
       ST_DWithin(boundary, p, buffer_m) AS in_ring
FROM zones, (SELECT ST_MakePoint(:lng,:lat)::geography p) q
WHERE ST_DWithin(boundary, p, 100);
```

---

3. Algorithms

3.1 Status thresholds (Available / Almost full / Full / Closed)

```
eff_cap   = capacity_car + floor(capacity_2w * pcu_weight_2w)     # or keep cars & 2W separate on the
UI
occ_s     = median(last 5 readings)  or EMA(α=0.4)                 # kill YOLO flicker
ratio     = occ_s / eff_cap

if manual_override in (closed, restricted) and (override_until is null or now < override_until):
status = override
elif schedule says closed now:                                     status = closed
elif now - occ_updated_at > 90 s:                                  status = unknown ("last updated 3
min ago")
elif ratio >= 1.00 or free <= 0:                                   status = full        (≥1.10 → raise
'overflow')
elif ratio >= 0.80:                                                status = almost_full
else:                                                              status = available
```

- **Hysteresis**, so the map doesn't blink: enter `almost_full` at ≥0.80 and leave only below 0.75. Enter `full` at ≥1.0 and leave below 0.95.
- **Small zones:** use absolute free slots too (e.g., ≤2 free ⇒ almost full), because 80% of 10 slots leaves two spaces.
- **Alerts** use a `dedupe_key` so each zone raises at most one alert per hour per kind.
- **"Closes in 30 min":** `next_close = min(schedule close today, next zone_closures.starts_at)`. If `0 < next_close - now ≤ 30 min`, set a `closing_soon` flag on the zone payload, show a banner on the citizen card, and push a WS `zone.closing_soon` event.

3.2 Grace-period dwell logic (possible illegal parking)

Per track, per ROI of role `no_parking` / `entrance_keepclear`, **or** any track inside a `count` ROI of a zone that is now `closed`:

```
moving ──(centroid moved < 0.03·frame_diag over last 10 s)──▶ stationary(since=t)
stationary ──(dwell ≥ grace)──▶ flagged  → backend creates incident(type=no_parking_violation,
                                          cctv_status='possible', evidence=snapshot crop)
any ──(not seen for > lost_timeout)──▶ gone   (if it was flagged → incident note "vehicle left at T")
```

- `grace`: real-world 5–10 min for no-parking, 2 min for keep-clear entrances or emergency access. For a closure, the grace is `zone_closures.grace_min` counted **from the closure time**, not from arrival. **Demo:** set it to 20–30 s via `DEMO_GRACE_SECONDS`.
- **ID-switch / occlusion tolerance:** when a new tracker id appears within 5 s with IoU ≥ 0.6 against a recently lost stationary track's `last_bbox`, inherit the old `stationary_since`. Otherwise one passing bus resets the timer. ByteTrack's `track_buffer` (frames kept after loss) helps too.
- **Debounce:** require the vehicle to be in the ROI for ≥ 80% of frames during the window. Ignore classes person/bicycle.
- **Human in the loop:** the system only creates a **"possible violation"**. An officer must press **Verify** (with optional plate entry) before any challan. This is the line to say for ethics, privacy and false-positive questions.

3.3 Duplicate complaint merging

Online (on each new complaint), the MVP path:

```
SELECT i.id,
       ST_Distance(i.location, :p) AS d_m,
       EXTRACT(EPOCH FROM (now() - i.last_activity))/60 AS dt_min
```

```
FROM incidents i
WHERE i.state IN ('open','assigned','verified')
  AND ST_DWithin(i.location, :p, 60)                    -- 60 m (GPS error in Indian streets is often
      10–30 m)
  AND i.last_activity > now() - INTERVAL '45 minutes'
ORDER BY i.location <-> :p LIMIT 5;
```

Score each candidate:

```
s = 0.40·(1 − d/60) + 0.25·(1 − dt/45) + 0.20·[category compatible] + 0.15·img_sim
img_sim = 1 − hamming(phash_new, phash_best_in_incident)/64     (0 if no photo)
merge if s ≥ 0.55  → complaints.incident_id = best, report_count += 1 (distinct reporter_hash),
recompute priority
else create new incident
```

- "Compatible" categories: double_parking ≈ illegal_parking ≈ no_parking_violation, and blocked_entrance ≈ blocked_footpath.
- **CCTV auto-link:** when a CV `possible violation` incident exists within 60 m / 45 min, the citizen report merges into it. That's the demo finale. And if a camera's `coverage` contains the complaint point, attach the latest snapshot and set `cctv_status='possible'`.
- **Perceptual hash:** `imagehash.phash(PIL.Image)` gives a 64-bit hash, and subtracting two hashes gives the Hamming distance. ≤10 means near-identical (same photo re-sent). ≤20 means likely the same scene. (imagehash) Note that the README's own example decodes the same JPEG with OpenCV and PIL and gets different hashes, so always hash through PIL (issue #89).
- **Stretch, CLIP similarity:** `open_clip` ViT-B/32 on CPU (~0.2–0.5 s/image; estimate, not benchmarked) → 512-d vector in pgvector. Use cosine ≥ 0.88 as a soft boost. Only worth it if there's time.
- **Batch / analytics** (re-cluster nightly or for the hotspot view): `ST_ClusterDBSCAN(ST_Transform(location::geometry, 32644), eps => 50, minpoints => 2) OVER (PARTITION BY date_trunc('hour', created_at))`. Add `ORDER BY` in the window for deterministic border assignment. (PostGIS ST_ClusterDBSCAN) For true spatio-temporal DBSCAN, use scikit-learn DBSCAN on `[x_m, y_m, t_min·k]`, where `k` converts minutes to metres (e.g., 1 min ≈ 2 m).

3.4 Priority score (0–100)

```
P = 100 · ( 0.30·S + 0.20·E + 0.15·R + 0.15·C + 0.10·A + 0.10·Z )

S  severity by type:   blocked emergency access 1.0, blocked entrance 0.8, double parking 0.7,
                       blocked footpath 0.6, no-parking 0.5, spillover 0.4, other 0.3
E  emergency route:    1 if within 100 m of hospital / fire station / ambulance route (OSM tags
                       amenity=hospital, amenity=fire_station precomputed into zones.near_emergency),
else 0
R  reports:            min(1, log2(1 + n_distinct_reporters) / log2(6))    → 5 reporters ≈ 1.0
(diminishing)
C  CCTV:               confirmed 1.0, possible 0.6, none 0
A  age (anti-starvation): min(1, minutes_open / 60)
Z  context:            0.5·[zone status full/overflow] + 0.5·[peak hour from profile]
```

- Show the **breakdown** in the incident drawer as chips ("+20 near hospital", "+15 CCTV"). Judges love explainability.
- Re-score every 60 s (APScheduler) and on every merge or verify. Sort the queue by P desc, with ties broken by first_seen.
- Optional reporter trust: multiply R by a reputation score (verified/total), so spam reports count less.

3.5 Hotspot detection

- **H3 hex binning** at resolution 9: average hex area ≈ 105,333 m² (~0.1 km²) and edge ≈ 201 m. Resolution 10 is ≈ 15,048 m² with a ≈ 76 m edge, a better match for a single street. (Values computed with `h3.average_hexagon_area/average_hexagon_edge_length` from h3-py.) Beach Road (10.918 N, 79.853 E) → res-9 cell `8961a949ecbffff`.
- **Weight:** `w = Σ exp(−age_days/14) · (1 + 0.5·verified)` per cell. Colour by quantiles. Flag a hotspot when w exceeds the 90th percentile **and** there are ≥ 3 distinct days. Optional significance test: Getis-Ord Gi* (PySAL `esda`), as a stretch.
- **KDE view:** feed verified incidents into `leaflet.heat` client-side for the smooth heatmap. H3 gives the "officer patrol suggestion" table (top 5 cells, with peak hour per cell).
- Seed 3–4 weeks of synthetic history (more on Market Street at weekend evenings, less elsewhere) so the heatmap isn't empty.

3.6 Occupancy forecasting & "usually full" insights

1. **Busy-period insight (do this first):** from `occupancy_profile` for today's weekday, find maximal runs of 15-min slots with `mean_ratio ≥ 0.9` (or p80 ≥ 0.95) lasting ≥ 1 h. Then merge and print "Usually full 6–9 PM". Cache it per zone per weekday.
2. **Short-horizon forecast (MVP, fast):** seasonal profile with level correction `ŷ(t+h) = profile(dow, slot(t+h)) · clip(current_ratio / profile(dow, slot(t)), 0.6, 1.5)`, then `predicted_full_at = first t+h (h ≤ 120 min) with ŷ ≥ 0.95` → "Likely full by 6:40 PM".
3. **LightGBM (stretch):** one global model across zones. Features: dow, slot, holiday/festival flag, zone_id (categorical), lag_15/30/60, rolling mean, weather if available. It trains in seconds. (LightGBM Python intro)
4. **Prophet (optional):** a `ds,y` dataframe per zone with daily + weekly seasonality. Fitting takes seconds per zone, but install weight (cmdstan) and per-zone fits make it the least hackathon-friendly option. (Prophet quick start)

- Evaluate with MAE on the last seeded week. Show one sentence and a sparkline on the police dashboard.

3.7 Spillover detection

- **Camera-side:** each `count` ROI gets a sibling `spillover_ring` ROI drawn just outside the painted boundary (road shoulder, footpath). A track that is stationary in the ring for > 2 min **while the zone ratio ≥ 0.9** raises a `spillover` alert and incident at the zone's edge point.
- **Geo-side:** citizen reports with `ST_DWithin(boundary, p, buffer_m) AND NOT ST_Covers(boundary, p)` are tagged spillover.
- **Overflow:** `occupied > eff_cap·1.1` (more cars counted than slots means double parking or parking outside the bays) → `overflow` alert.

---

4. APIs

Conventions: JSON, ISO-8601 UTC timestamps, GeoJSON for geometry. Officer routes take a `Bearer` JWT. Ingest routes take an `X-Worker-Key` header.

4.1 REST

Method & path	Who	Purpose
`GET /health`	all	liveness + DB ping
`GET /zones?bbox=&status=`	citizen	GeoJSON FeatureCollection: status, free, capacity, timings, `closing_soon`, `busy_insight`, `predicted_full_at`

Method & path	Who	Purpose
`GET /zones/nearby?lat=&lng=&limit=10&vehicle=car|2w`	citizen	sorted by distance, with nav deep link
`GET /zones/{id}`	all	detail + today's profile curve
`GET /zones/{id}/history?from=&to=&bucket=15m`	police	occupancy series
`GET /zones/{id}/forecast?horizon=120`	all	forecast points
`POST /zones` / `PATCH /zones/{id}`	admin	create/edit polygon, capacity, rules
`POST /zones/{id}/status` {action: open|close|restrict, reason, until?, grace_min?}	police	one-click control → audit + WS
`POST /zones/{id}/closures`	police	schedule a future closure (festival)
`GET /cameras` · `POST /cameras` · `PUT /cameras/{id}/rois`	admin	camera + pixel ROI config (worker pulls this)
`GET /cameras/{id}/snapshot.jpg` · `GET /cameras/{id}/mjpeg`	police	latest annotated frame / live MJPEG preview
`POST /ingest/occupancy` [{zone_id, occupied, occupied_2w, camera_id, ts, confidence}]	worker	batched counts every 1–2 s
`POST /ingest/tracks` [{camera_id, session_id, tracker_id, roi_id, cls, bbox, stationary_since, state}]	worker	track state changes only
`POST /ingest/violation` (multipart: json + snapshot.jpg)	worker	dwell exceeded → incident
`POST /ingest/heartbeat`	worker	camera fps, lag → offline alert
`POST /complaints` (multipart: photo, lat, lng, accuracy, category, desc)	citizen	returns {complaint_id, incident_id, merged: bool, merge_score}
`GET /complaints/{id}`	citizen	status tracking ("Verified by officer")
`GET /incidents?state=open&sort=priority`	police	priority queue
`GET /incidents/{id}`	police	merged complaints, evidence, priority breakdown, timeline
`POST /incidents/{id}/assign` {officer_id}	supervisor	
`POST /incidents/{id}/verify` {plate?, note, photo?} · `/dismiss` · `/resolve`	officer	audit logged
`POST /incidents/{id}/merge` {other_id}	officer	manual merge override
`POST /incidents/{id}/echallan`	officer	**stub**: builds a payload and stores echallan_ref='DRAFT-…'
`GET /alerts?since=` · `POST /alerts/{id}/ack`	police	
`GET /analytics/hotspots?res=9&days=30`	police	H3 cells + weights (GeoJSON)
`GET /analytics/heat?days=30`	police	[[lat,lng,w],...] for leaflet.heat
`GET /analytics/kpis`	police	total capacity, occupancy %, open incidents, avg response time

Method & path	Who	Purpose
`POST /auth/login`	officer	JWT
`GET /audit?entity=`	admin	
`POST /demo/simulate-vehicle {zone_id|roi_id, action: park|leave, count}`	demo	injects synthetic tracks through the **same** ingest path
`POST /demo/reset` · `POST /demo/step/{n}`	demo	reset the seed and jump to scripted states (backup plan)

4.2 WebSocket `/ws?role=citizen|police` (server → client events)

```
{ "type": "zone.updated",       "data": { "id": 3, "status": "almost_full", "occupied": 17,
"capacity": 20, "free": 3, "ts": "..." } }
{ "type": "zone.status_changed","data": { "id": 3, "from": "available", "to": "closed", "reason":
"Festival", "by": "PC-1142" } }
{ "type": "zone.closing_soon",  "data": { "id": 3, "closes_at": "...", "minutes": 30 } }
{ "type": "alert.created",      "data": { "id": 91, "kind": "possible_illegal", "level": "warn",
"zone_id": 3, "incident_id": 55 } }  // police only
{ "type": "incident.created",   "data": { "id": 55, "type": "no_parking_violation", "priority": 72.4,
"cctv_status": "possible", "lat": ..., "lng": ... } }
{ "type": "incident.updated",   "data": { "id": 55, "state": "verified", "report_count": 2,
"priority": 84.1 } }
{ "type": "complaint.merged",   "data": { "complaint_id": 210, "incident_id": 55, "score": 0.81 } }
{ "type": "camera.status",      "data": { "id": 1, "online": true, "fps": 3.1 } }
```

Client → server: `{"type":"subscribe","bbox":[...]}` (optional) and `{"type":"ping"}` every 25 s. The ping also keeps Render's free instance awake.

---

5. Demo script (6 minutes) + pitch structure

5.1 Setup (done before walking on stage)

- Laptop running `docker compose up` with postgis, api, mediamtx (cam1 = Beach Road loop, cam2 = no-parking stretch loop), worker, and the web app.
- Browser window A (big screen): **Police dashboard** at `/police`. Window B: CCTV view, an annotated MJPEG with zone polygons + counts. Phone (mirrored with `scrcpy` or a second browser at mobile width): **Citizen map** at `/`.
- Seeded data: **Karaikal** zones along Beach Road and nearby streets, plus a second cluster in **Puducherry** near the Beach promenade/Goubert Avenue. That shows multi-city and Puducherry UT scale. Reference points: Karaikal Beach ≈ 10.918 N, 79.853 E (Wikipedia). A mapped "Beach Car Parking" at ≈ 10.91424 N, 79.84987 E (listing). Puducherry Beach ≈ 11.93492 N, 79.83594 E (2markers). Beach Road (Karaikal) is ~2.94 km long (GeoIQ). **These coordinates are approximate. Draw the real polygons in geojson.io over the OSM base layer.**
- Example seeded zones: `Beach Road — North (20 car / 30 2W)`, `Beach Road — Lighthouse end`, `Bharathiyar Road lot`, `Market Street (no-parking 8–11 AM)`, `Hospital entrance keep-clear`. Add 4 weeks of synthetic snapshots with an evening peak (6–9 PM) and a Sunday peak.
- `DEMO_GRACE_SECONDS=25`. Second phone pre-loaded with a citizen report photo of the same car (or take it live).

5.2 Minute-by-minute

Time	On screen	Say (gist)	Mechanics / fail-safe
0:00–0:40	Title → one photo of a jammed Indian street	**Problem:** drivers circle looking for legal parking. Police have no single view of	Static slide.

Time	On screen	Say (gist)	Mechanics / fail-safe
		which streets are full, where illegal parking is, or which rules changed today. The CCTV is already on the poles.	
0:40–1:30	CCTV window: boxes + IDs, zone polygon, counter “Beach Road: 16/20”	**Step 1, CCTV counts.** Existing camera, YOLO + tracking on a normal laptop CPU, no sensors in the ground. Each painted bay area is a polygon.	Looped MP4 → MediaMTX RTSP. If the CV hiccups, the counter keeps the last value. Say “3 fps is plenty for parking”.
1:30–2:15	Phone: citizen map, Beach Road **amber “4 left”**. Tap → card: timings, “Usually full 6–9 PM”, “Likely full by 6:40 PM”, **Navigate**	**Step 2, citizen map.** Nearby parking, free spaces, timings, busy-hour insight from history, one tap to Google Maps navigation.	Press +1 on the hidden simulate panel so the count ticks to 17 live and the judges see realtime.
2:15–2:55	Police dashboard: KPI tiles, live map, alert feed shows “Beach Road almost full”	**Police view.** Every street at a glance, plus alerts for almost full, overflow, possible violations.	Alert was triggered by the 17/20 tick (≥0.80).
2:55–3:30	Officer clicks **Close → Reason: Festival procession → 15-min leave window**. Phone map **instantly turns grey “Closed – Festival”** with a banner	**Steps 3–4, police close the street → map changes instantly.** Every action is audit-logged with officer badge + reason.	WS `zone.status_changed`. Backup: the phone polls `/zones` every 10 s, so it updates within 10 s even if the WS drops.
3:30–4:20	CCTV: most cars leave (loop segment) but one car stays; label shows dwell timer `00:18…` `00:25` → turns red. Dashboard: **“Possible illegal parking – Beach Road”** pops into the priority queue with snapshot	**Steps 5–6, a vehicle remains → flagged after the grace period.** The demo grace is 25 s; in real life it’s configurable (e.g., 10 min). The system never fines anyone. It flags for a human.	If the real video doesn’t show a car remaining, click **Simulate vehicle → park in closed zone**. It goes through the same ingest + dwell code.
4:20–4:50	Second phone: Report → “Double parking”, photo, GPS auto → Submit → toast **“Added to existing case #55 (2 reports)”**. Dashboard incident card now shows **CCTV snapshot + citizen photo**, priority jumps 62 → 81 with chips	**Step 8, citizen report merged into the same incident.** Same place (within 60 m), same time window, same vehicle type. No duplicate work for police. CCTV corroboration plus a citizen photo means higher priority.	Pre-position the phone GPS near the seeded camera coverage. If GPS is off indoors, allow “drop pin” fallback (the report form has a map picker).
4:50–5:20	Officer clicks **Verify** (types plate) → state “Verified” → **“Send to eChallan (draft)”** button	**Step 7, officer verifies.** Human in the loop. Future: hand verified cases to the national eChallan workflow (MoRTH/NIC). We only stub the payload; MoRTH’s data-sharing policy routes API access through the NIC API gateway with approval.	Stub returns `DRAFT-…`. Don’t claim live integration.
5:20–5:45	Toggle **Hotspots** layer (H3 hexes + heat) and a forecast sparkline	**Add-ons:** repeat hotspots for patrol planning, spillover detection, “full by” forecasts.	Seeded history.
5:45–6:30	Slide: impact / scale / cost	**Impact, scalability, cost** (5.4 below). Close with “uses	

Time	On screen	Say (gist)	Mechanics / fail-safe
		cameras the city already owns".	

5.3 Making it reliable (fake what's risky, keep the logic real)

1. **Video:** a pre-recorded MP4 looped via `ffmpeg -re -stream_loop -1 … rtsp://…/cam1`. Pick or trim a clip where the "car stays" moment happens within ~60 s of a loop point, or cut it so it does. Keep the clip short (60–90 s) so the loop is predictable.
2. **One code path:** the `Simulate vehicle` and `Simulate ±1` buttons (hidden police-side panel, `Shift+D`) call `/demo/*`, which writes through `/ingest/*`. Same rules engine, honest demo.
3. **Scripted state machine:** `POST /demo/step/{1..6}` sets the DB to each storyboard state (counts, closure, flagged vehicle, merged report). If anything goes wrong live, the presenter presses the next step key and the UI follows over WebSocket.
4. **Offline-first:** everything runs on the laptop. The phone joins the laptop hotspot. No dependency on venue Wi-Fi, Render wake-up, or the OSRM demo server. Cache OSM tiles for the demo area: pan around once beforehand so the browser cache has them, or bundle a small offline tile set.
5. **Recorded backup:** a 2-minute screen recording of the full flow (OBS) on the desktop and on a USB stick. If live fails twice, play it and narrate.
6. **Pre-flight checklist (T-10 min):** `/demo/reset`, cameras green, phone GPS on, Render URL warmed (if showing the cloud), brightness up, notifications off, second phone photo ready.

5.4 Judging pitch structure (with numbers to fill in)

- **Problem (30 s):** cruising for parking, footpath blocking, police blind spots. Cite your city observation (e.g., the Beach Road evening rush). Avoid unsourced statistics.
- **Solution (60 s):** one platform, three users (citizen, police, planner), built on existing CCTV.
- **Live demo (3.5 min):** 5.2 above.
- **Impact:** less search time and congestion, faster response to blocked entrances and emergency routes, data for planners (busy hours, hotspots), transparent and audit-logged enforcement with a human in the loop.
- **Alignment:** MoHUA/MoUD TMICC operations guidance lists **parking information dissemination** (location, timing, changes) as a TMICC function, and lists parking violations and handheld e-Challan under enforcement (Guwahati TMICC operations document, MoHUA). ICCCs exist in all 100 Smart Cities (PIB). eChallan is MoRTH's digital enforcement platform, integrated with Vahan/Sarathi (echallan.parivahan.gov.in, user manual). External API access goes via the NIC API gateway with MoRTH approval (data sharing policy).
- **Scalability:** stateless API + Postgres. CV workers scale horizontally (one worker per 4–8 cameras at 2–3 fps on a CPU box, or a single GPU box for 50+ cameras; estimates). Plugs into an ICCC as a microservice. Zone config is just polygons, so a new street takes minutes.
- **Cost:** no per-bay sensors (Raipur's smart-parking RFP, for comparison, specifies magnetic/IR/ultrasonic sensors (RFP)). Open-source stack, ₹0 hosting for the pilot, and the main cost is compute for existing cameras.
- **Risks & answers:** privacy (no face/plate recognition in MVP, plates entered only by an officer after verification, snapshots auto-deleted after N days), false positives (grace period + human verification), night/rain (confidence score + "estimated" status), Ultralytics AGPL licence for production.

---

6. Build plan (team of 3–4, 24–48 h)

6.1 Roles

- **A: CV lead** (Python): MediaMTX/ffmpeg, worker, ROI tool, dwell state machine, evidence crops, MJPEG preview.

- **B: Backend lead** (the builder): schema + seeds, FastAPI ingest/rules/WS, merge + priority, demo endpoints, deploy.
- **C: Citizen frontend:** map, zone cards, nearby list, nav links, PWA report flow, merge toast.
- **D: Police dashboard + data/pitch:** dashboard UI, alerts, incident drawer, hotspots/forecast views, synthetic history generator, slides, backup video. In a 3-person team, D's work splits between C (UI) and B (data).

6.2 Timeline (48 h; compress by ~half for 24 h by dropping the stretch column)

Hours	A (CV)	B (Backend)	C (Citizen)	D (Police/Data/ Pitch)
0–2	Pick or record clips, MediaMTX streaming	Repo, docker-compose (postgis, api), schema v1, agree **API contract + WS event JSON** (freeze it)	Vite+React+Tailwind+shadcn scaffold, react-leaflet map	Draw Karaikal/ Puducherry polygons in geojson.io, seed script
2–8	YOLO-n + ByteTrack on RTSP, PolygonZone counts, POST `/ingest/occupancy`	Zones CRUD, `/zones` GeoJSON, status engine + hysteresis, WS broadcast	Zones coloured by status, zone card, nearby list, Google Maps deep link	Police layout: KPIs, map, alert feed (mock data from contract)
8–14	Stationary/dwell state machine, snapshot crops, `/ingest/violation`	Closures + schedule, `closing_soon`, audit log, `/zones/{id}/status`, `/demo/simulate-vehicle`	WS hook + live updates, closed/closing banners	Close/Open/Restrict controls, incident queue, drawer
14–20	ROI config pulled from API, MJPEG annotated preview endpoint	Complaints API + uploads + phash, **merge algorithm**, priority score + scheduler	Report flow: camera, GPS, category, upload, merge toast; PWA manifest	Synthetic 4-week history generator → busy insight + profile
20–24	Hardening: reconnect, frame-drop thread, heartbeat	**End-to-end demo flow working locally** (MVP gate)	Polish mobile UX	Evidence viewer (CCTV + citizen photos), verify/ dismiss
24–30	Spillover ring ROI, ID-switch tolerance	Forecast (profile-ratio), hotspots API (H3), eChallan stub	Forecast "likely full by", busy-hours line	Hotspot layer (H3 + heat), forecast sparkline
30–38	Tune thresholds on the clips, record CV-only backup clip	Deploy: Neon + Render + Vercel; `/demo/step/*` scripted states	Offline queue (stretch), i18n Tamil labels (stretch)	Slides, pitch script, record full backup video
38–44	Rehearsal support	Bug bash, load seed, freeze	Bug bash	**3 full rehearsals** with timer
44–48	Buffer / sleep	Buffer	Buffer	Buffer

Integration rule: freeze the JSON contract at hour 2 and have every UI start on mock JSON. Integrate at hours 8, 14 and 20. "Everything works end-to-end locally" is the hard gate at hour 24.

6.3 MVP vs stretch

MVP (must demo): - 1–2 simulated cameras → live counts → zone status with hysteresis - Citizen map: status colours, free count, timings, nearby list, Google Maps navigation, closing-soon banner, busy-hours line (from seeded history) - Police: live map + KPIs, one-click open/close/restrict with reason, alert feed (almost full / overflow / possible illegal) - Dwell → possible violation incident with CCTV snapshot; officer

verify/dismiss; audit log - Citizen report (photo + GPS) → **merge into the existing incident** (ST_DWithin + time + category) → priority queue - Demo controls: simulate vehicle, reset, scripted steps

Stretch (in order of judge value per hour): 1. Priority breakdown chips (cheap, high value) 2. H3 hotspot layer + heatmap 3. “Likely full by” forecast (profile-ratio) → LightGBM 4. Spillover ring detection 5. pHash image similarity in merge → CLIP + pgvector 6. eChallan draft payload + PDF case sheet 7. PWA offline queue, Tamil/English toggle 8. Officer mobile view with “nearest incidents” (geo-sorted by officer location) 9. Homography: plot detected vehicles as dots on the map 10. Camera-offline alerts, Prometheus metrics (MediaMTX exposes them)

---

7. Open-source starters, libraries & references

Repo / resource	What to take from it	Status
ultralytics/ultralytics	YOLO detection + tracking (ByteTrack/ BoT-SORT/…); `solutions.ParkingManagement`, `ParkingPtsSelection`	Opened; AGPL-3.0, ~62k stars
Ultralytics parking guide · track mode	Exact code for slot occupancy and `persist=True` tracking	Opened
roboflow/supervision – time_in_zone example	PolygonZone + per-track timers + `draw_zones.py` polygon tool. **Closest to the dwell/grace feature**	Seen via search (repo pages + docs)
supervision docs: count in zone	`sv.PolygonZone`, `zone.trigger()`	Search excerpt
bluenviron/mediamtx + docs	Simulated RTSP cameras from MP4	Opened; MIT, ~20k stars
fastapi/full-stack-fastapi-template	FastAPI + SQLModel + Postgres + React/Vite + Tailwind + shadcn + JWT + Docker Compose. Fork it, or copy the auth and project layout	Search listing (MIT, ~45k stars)
FastAPI WebSockets docs	ConnectionManager broadcast pattern	Search excerpt
shashibhat/SmartPark	Tiny reference: FastAPI + WS + `/ingest/slots` adapter for Ultralytics ParkingManagement. Good API-shape inspiration, not production code	Opened; 0 stars, 3 commits
danielbob32/ParkingSpace	YOLOv11 + RTSP parking detection, works in areas **without painted bays** (relevant to Indian streets)	Search only
MostafaAttia-creator/parking-slots-detector-and-tracking	YOLO11 + ByteTrack + parking duration (Streamlit)	Search only
Leaflet/Leaflet.heat	Heatmap layer	Search listing (BSD-2)
uber/h3-py · uber/h3-js	Hex binning server and client side	Search listing; h3-py installed and run in sandbox
JohannesBuchner/imagehash	pHash for duplicate photos	Search listing (BSD-2)
vite-pwa/vite-plugin-pwa	PWA for the citizen app	Search listing
Project-OSRM/osrm-backend · HTTP API	Route preview (self-host with Docker for production)	Search listing
PostGIS ST_ClusterDBSCAN	In-DB clustering	Search excerpt
CNRPark+EXT · PKLot	Validation images for occupancy accuracy	Search excerpts
Prophet · LightGBM	Forecasting	Search excerpts

Repo layout suggestion

```
smart-parking/
  docker-compose.yml        # postgis, api, mediamtx, worker, web
  mediamtx/mediamtx.yml     # runOnDemand ffmpeg loops for cam1..camN
  media/clips/*.mp4
  api/  (FastAPI: routers/ zones, ingest, complaints, incidents, analytics, demo, ws; services/
status, dwell_rules, merge, priority, forecast, hotspots)
  worker/ (capture.py, detect_track.py, zones.py, dwell.py, client.py, config pulled from /cameras)
  web/  (Vite React: routes/citizen, routes/police, components/map, hooks/useLiveZones)
  seed/ (zones_karaikal.geojson, zones_puducherry.geojson, gen_history.py)
```

---

8. Gaps & caveats (be honest in Q&A)

- **Performance numbers** (CPU fps, cameras per worker, CLIP latency) are estimates and were not benchmarked here. Measure on the actual laptop with `yolo26n`/`yolo11n` at `imgsz=640` and quote what you get.
- **Zone coordinates** listed above are approximate POI points from public listings. Draw the real polygons yourself over OSM.
- **eChallan** has no public self-serve API. The integration is a future, approval-gated hook, so present it as a stub.
- **OSM tile usage policy** was not checked in this research. Light demo traffic is normal practice, but for production use a tile provider or self-hosted tiles.
- Indian street parking is often **unmarked and mixed (2W + cars)**. Count both classes and make capacity per vehicle type configurable. Don’t promise slot-level accuracy on unmarked streets: zone-level counts are the honest metric.